

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_b7f012f8-5c3\agent-a95ed9a0f7516b8f7.jsonl`  
- SHA-256: `0f70e9f1549910ba6e74d5aab25b42b8072fc3fd2720c7ad1973519c53c50ab5`  
- Source modified: `2026-06-18T01:30:26+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_b7f012f8-5c3`  
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T01:27:48.619Z)

You are an adversarial bug-hunter in a mature Python repo (badminton-highlight-indexer: FastAPI + SQLite + ffmpeg + GPU CV/AI rally detector). Read code READ-ONLY in this worktree (pinned to origin/master ? DO NOT modify anything):

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base

Hunt ONLY for REAL, REACHABLE defects a maintainer would agree is a bug and want fixed:

- crash on a reachable input (unguarded dict/index/attr access, None deref, KeyError/IndexError),
- wrong logic / off-by-one / inverted condition / wrong operator,
- incorrect error handling (catching too much/little, swallowing, wrong exception type),
- resource leak (unclosed file/db/subprocess), TOCTOU, mutable-default-arg,
- contract mismatch between a function and its callers/tests,
- wrong default / unit / sign that produces incorrect output.

HARD RULES:

- PRECISION over recall. Report a finding ONLY if you traced the actual code and a concrete trigger reaches it. No style, no "could be cleaner", no speculative "might", no perf-nits, no missing-test observations, no naming. If you are not sure it is a real bug, DROP it.
- For each finding give an EXACT file path + line, the bug_class, a CONCRETE trigger (the input or call path that hits it), why it is wrong, severity, and a minimal proposed fix.
- This repo has 848 passing tests and several hardening sweeps ? easy bugs are already gone. Expect FEW or ZERO real bugs in your area. Returning an empty candidates list is a VALID, respected answer. Do NOT manufacture findings to look productive.
- EXCLUDE these files entirely (just fixed / fix already in flight): backend/eval/ablation.py, backend/eval/fusion_compare.py, backend/eval/fusion_audio.py.
- Verify your trigger by reading the surrounding code and any related test, not from memory.

YOUR AREA: backend/providers/, backend/sports/, backend/annotations/, backend/reporting/, backend/tools/. Focus on the provider/registry interfaces, Gemini/cloud-provider calls + error/retry handling, response parsing, and any contract mismatch with callers.

2. assistant (2026-06-18T01:27:51.493Z)

I'll start by exploring the structure of my assigned area to understand what I'm working with.

3. user (2026-06-18T01:27:53.452Z)

4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
Not currently on any branch.
nothing to commit, working tree clean

4. user (2026-06-18T01:27:53.852Z)

```
backend\providers\__init__.py
backend\providers\base.py
backend\providers\gemini.py
```

5. user (2026-06-18T01:27:54.489Z)

```
backend\sports\__init__.py
backend\sports\badminton.py
backend\sports\base.py
```

6. user (2026-06-18T01:27:54.583Z)

```
backend\annotations\base.py
backend\annotations\file_provider.py
backend\annotations\automated_provider.py
backend\annotations\__init__.py
```

7. user (2026-06-18T01:27:55.017Z)

```
backend\reporting\harvest.py
backend\reporting\run_signals.py
backend\reporting\__init__.py
```

8. user (2026-06-18T01:27:55.535Z)

```
backend\tools\rally_slicer.py
backend\tools\__init__.py
backend\tools\export.py
```

9. assistant (2026-06-18T01:27:58.258Z)

Let me read the core provider files first.

10. user (2026-06-18T01:27:59.169Z)

```
1      import os
2      import time
3      import json
4      import logging
5      from typing import Tuple
6      from google import genai
7      from google.genai import types
8      from backend.providers.base import AIVideoProvider
9      from backend.infrastructure.execution_policy import (
10          ExecutionPolicy, PolicyTimeout, poll_until, run_bounded,
11      )
12
13      logger = logging.getLogger(__name__)
14
15      # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
16      # each uploaded clip is below this. Oversize uploads are refused (not silently
17      # attempted).
18      MAX_UPLOAD_MB = 500
19
20      class GeminiProvider(AIVideoProvider):
21          def __init__(self, api_key: str, model_name: str):
22              super().__init__(api_key, model_name)
23              # Initialize client lazily to avoid connection/auth issues during class
24              # instantiation
25              self._client = None
26
27          @property
28          def client(self):
29              if self._client is None:
```

```

28         self._client = genai.Client(api_key=self.api_key)
29     return self._client
30
31     def _safe_delete(self, video_file) -> None:
32         """Best-effort cleanup of an uploaded File (never raises)."""
33         try:
34             self.client.files.delete(name=video_file.name)
35         except Exception:
36             pass
37
38     def analyze_video(self, video_path: str, prompt: str,
39                      chunk_duration: float = 0.0,
40                      label: str = "") -> Tuple[list, dict]:
41         log_prefix = f"Gemini {label}" if label else "Gemini"
42
43         metrics = {
44             "upload_time": 0.0,
45             "process_time": 0.0,
46             "gen_time": 0.0
47         }
48
49         # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
50         # safety/speed)
51         try:
52             size_mb = os.path.getsize(video_path) / (1024 * 1024)
53         except OSError:
54             size_mb = 0.0
55         if size_mb > MAX_UPLOAD_MB:
56             logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
57             {MAX_UPLOAD_MB}MB cap ? "
58             f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
59             f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
60             chunk smaller.")
61             metrics["oversize_skipped"] = True # callers aggregate this into the run
62             report
63             return [], metrics
64
65         # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
66         # connections)
67         logger.info(f"[{log_prefix}] Uploading {video_path}...")
68         t_upload_start = time.time()
69         video_file = None
70         for attempt in range(3):
71             try:
72                 video_file = self.client.files.upload(file=video_path)
73                 metrics["upload_time"] = time.time() - t_upload_start
74                 logger.info(f"[{log_prefix}] Upload complete in
75                 {metrics['upload_time']:.1f}s. URI: {video_file.uri}")
76                 break
77             except Exception as e:
78                 logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
79                 {e}")
80                 if attempt < 2:
81                     time.sleep(2 * (attempt + 1))
82         if video_file is None:
83             logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
84             return [], metrics
85
86         # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
87         # not hang
88         # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
89         # EXECUTION_POLICY.md.
90         t_process_start = time.time()
91         process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0) #
92         20 min cap

```

```

83
84     def _processed():
85         nonlocal video_file
86         video_file = self.client.files.get(name=video_file.name)
87         return None if video_file.state.name == "PROCESSING" else video_file
88
89     try:
90         video_file = poll_until(_processed, process_policy,
91                                label=f"{log_prefix} processing", logger=logger)
92     except PolicyTimeout:
93         logger.error(f"[{log_prefix}] Processing exceeded
94 {process_policy.max_wait_sec:.0f}s ? giving up.")
95         self._safe_delete(video_file)
96         return [], metrics
97     except Exception as e:
98         logger.error(f"[{log_prefix}] Status check failed: {e}")
99         self._safe_delete(video_file)
100        return [], metrics
101    if video_file.state.name == "FAILED":
102        logger.error(f"[{log_prefix}] Video processing failed:
103 {video_file.error.message}")
104        self._safe_delete(video_file)
105        return [], metrics
106
107    metrics["process_time"] = time.time() - t_process_start
108    logger.info(f"[{log_prefix}] Processing finished in
109 {metrics['process_time']:.1f}s.")
110
111    # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-
112    # killer that was
113    # missing: a generate() that never returns previously blocked forever) + retry
114    # the 503/network
115    # FAILURE waves (capacity 503s on the flagship flash come in bursts; ~6
116    # attempts).
117    logger.info(f"[{log_prefix}] Submitting multimodal request to
118 {self.model_name}...")
119    t_gen_start = time.time()
120    gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
121 backoff_base_sec=3.0)
122    raw_text = None
123
124    def _generate():
125        return self.client.models.generate_content(
126            model=self.model_name,
127            contents=[video_file, prompt],
128            config=types.GenerateContentConfig(
129                response_mime_type="application/json", temperature=0.0),
130        )
131
132    try:
133        response = run_bounded(_generate, gen_policy, label=f"{log_prefix}
134 generate", logger=logger)
135        raw_text = response.text
136        metrics["gen_time"] = time.time() - t_gen_start
137        logger.info(f"[{log_prefix}] Response received in
138 {metrics['gen_time']:.1f}s:\n{raw_text}")
139    except Exception as e:
140        # PolicyTimeout (hang) or the last 503 ?
141        both end here
142        logger.error(f"[{log_prefix}] generate gave up after retries: {e}")
143        self._safe_delete(video_file)
144
145    if not raw_text:
146        return [], metrics
147
148    # 4. Parse JSON

```

```

137         try:
138             segments = json.loads(raw_text)
139             if not isinstance(segments, list):
140                 logger.error(f"[{log_prefix}] Response is not a JSON array.")
141                 return [], metrics
142             return segments, metrics
143         except Exception as e:
144             logger.error(f"[{log_prefix}] JSON parse failed: {e}")
145             return [], metrics
146

```

11. user (2026-06-18T01:27:59.805Z)

```

1     from abc import ABC, abstractmethod
2     from typing import Tuple
3
4     class AIVideoProvider(ABC):
5         def __init__(self, api_key: str, model_name: str):
6             self.api_key = api_key
7             self.model_name = model_name
8
9         @abstractmethod
10        def analyze_video(self, video_path: str, prompt: str,
11                          chunk_duration: float = 0.0,
12                          label: str = "") -> Tuple[list, dict]:
13            """Sends a video file to the AI model along with a prompt, and returns a tuple
14            of
15            (segments_list, metrics_dict).
16
17            Args:
18                video_path: Path to the video file to analyze.
19                prompt: Prompt instructions.
20                chunk_duration: Duration of this chunk in seconds, or 0.0 if not chunked.
21                label: Diagnostic label for logging.
22
23            Returns:
24                Tuple containing:
25                - A list of detected segment dicts (e.g. [{"start_time": ..., "end_time":
26                ...}])
27                - A dictionary of performance metrics (e.g. {"upload_time": ...,
28                "process_time": ..., "gen_time": ...})
29            """
30            pass
31

```

12. assistant (2026-06-18T01:28:03.534Z)

Now let me read the remaining files in my area.

13. user (2026-06-18T01:28:04.967Z)

```

1     from backend.sports.base import SportProfile
2
3     class BadmintonSportProfile(SportProfile):
4         @property
5         def name(self) -> str:
6             return "badminton"
7
8         def get_prompt(self, chunk_duration: float = 0.0) -> str:
9             base_prompt = """System Role & Objective
10            You are a highly precise computer vision data extraction engine specializing in
11            badminton match analysis. Your exclusive objective is to parse a full-length match video and
12            generate a flawless JSON array containing exact rally boundaries and shot metrics for automated
13            highlight generation.

```

```
11
12     Core Directives
13     Detect every single active rally in the video. Do not miss any live gameplay.
14     Extract high-precision timestamps as TOTAL DECIMAL SECONDS from the start of the video
for exact frame-accurate cuts.
15     Estimate the number of shots exchanged (net crossings) and provide a confidence score
for that count, acknowledging frame-sampling limitations.
16
17     CRITICAL TIMESTAMP FORMAT RULES:
18     - All start_time and end_time values MUST be expressed as TOTAL SECONDS from the
beginning of the video, as a decimal number.
19     - Example: A rally starting at 1 minute 14 seconds into the video MUST be written as
74.0, NOT as "1:14" or 114.
20     - Example: A rally starting at 2 minutes 30.5 seconds MUST be written as 150.5, NOT as
"2:30.5" or 230.5.
21     - NEVER use MM:SS or HH:MM:SS colon-separated format. NEVER concatenate minutes and
seconds into a single number without converting.
22     - The downstream pipeline will BREAK if timestamps are not in total decimal seconds.
23
24     Positive Constraints: Rally Boundaries
25     Start Condition: The exact fractional second the server's racket makes contact with the
shuttlecock.
26     End Condition: The rally concludes ONLY when both of the following physical criteria are
met sequentially: First, the shuttlecock hits the court surface, strikes the net and drops, or
lands visibly out of bounds. Second, a 0.3 to 0.5-second buffer has elapsed where both players
have visibly ceased all kinetic momentum (no lunging, sliding, or racket follow-through).
27
28     Negative Constraints: Anomaly Rejection (CRITICAL)
29     Replay Rejection: You must strictly ignore and filter out all slow-motion instant
replays. Replays are not live rallies.
30     Dead-Time Rejection: Ignore all Hawkeye/line-call challenges, interval coaching, towel
breaks, court mopping, changing shuttles, and player celebrations.
31     Pre-serve Rituals: Do not start the timestamp during the players' setup or stance. Wait
for physical racket-to-shuttle contact.
32
33     Shot Tracking & Data Confidence
34     Shot Definition: Count every successful traverse of the shuttlecock across the net
(serves, clears, smashes, drops, drives). Do not count the final grounding or pre-net faults.
35     Confidence Scoring: Because the shuttlecock moves at extreme velocities that may fall
between sampled frames, provide a shot_count_confidence metric (Low, Medium, High). Base your
estimate on player biomechanics, positioning, and racket swings if the shuttle itself is
blurred.
36
37     Strict Output Schema
38     Output strictly in the following JSON array format. Do not include markdown formatting,
introductory text, conversational filler, or explanations. Any deviation from this schema will
break the downstream pipeline.
39     Remember: start_time and end_time are TOTAL SECONDS (decimal), not MM:SS.
40
41     [
42     {
43     "start_time": 74.0,
44     "end_time": 81.3,
45     "shuttle_exchange_count": 11,
46     "shot_count_confidence": "High"
47     },
48     {
49     "start_time": 150.5,
50     "end_time": 174.2,
51     "shuttle_exchange_count": 24,
52     "shot_count_confidence": "Medium"
53     }
54     ]
55     ""
56         if chunk_duration > 0.0:
```

```

57         duration_context = (
58             f"IMPORTANT VIDEO CONTEXT: This video segment is exactly
{chunk_duration:.1f} seconds long. "
59             f"All timestamps MUST be between 0 and {chunk_duration:.1f}. "
60             f"Any timestamp beyond {chunk_duration:.1f}s is INVALID.\n\n"
61         )
62         return duration_context + base_prompt
63     return base_prompt
64
65     def get_relevance_config(self) -> dict:
66         return {
67             "court_green_lower": [35, 40, 40],
68             "court_green_upper": [85, 255, 255],
69             "court_pixels_min_ratio": 0.05
70         }
71

```

14. user (2026-06-18T01:28:05.457Z)

```

1     from abc import ABC, abstractmethod
2
3     class SportProfile(ABC):
4         @property
5         @abstractmethod
6         def name(self) -> str:
7             """The identifier of the sport (e.g., 'badminton')."""
8             pass
9
10        @abstractmethod
11        def get_prompt(self, chunk_duration: float = 0.0) -> str:
12            """Full system prompt for the AI model, including output schema."""
13            pass
14
15        @abstractmethod
16        def get_relevance_config(self) -> dict:
17            """Sport-specific visual validation params (court colors, thresholds, etc.)."""
18            pass
19

```

15. user (2026-06-18T01:28:06.378Z)

```

1     """AutomatedProvider ? the fast-tier "oracle" annotation source (GS-4, interface stub).
2
3     The fast/CI tier of the golden-set flow needs an *automated* labeler that turns a
4     video into rally `(start, end)` intervals quickly and cheaply, as a smoke alarm
5     between authoritative human-vendor runs (the slow tier).
6
7     This slice ships the **interface only**, deliberately model-agnostic: an
8     `AutomatedProvider` that delegates labeling to an injected `labeler` callable, plus
9     a deterministic fake for tests/CI. No concrete model is wired yet ? that's a
10    tracked backlog item (docs/BACKLOG.md), to be chosen alongside the production LLM
11    step so the lineages differ (see the oracle rule below).
12
13    ?? **Oracle rule.** The labeler MUST be a *different model lineage* than whatever
14    production uses. If production becomes the offline ActionFormer segmenter and the
15    oracle is also ActionFormer, they share blind spots and the regression test is
16    blind ? green while both are wrong. `lineage` records the oracle's model family so
17    callers can guard against it matching production (`warn_if_same_lineage`).
18    """
19    import logging
20    from typing import Callable, Dict, List, Optional
21
22    from backend.annotations.base import (
23        AnnotationProvider,
24        annotation_registry,

```

```

25     Interval,
26     DONE,
27     FAILED,
28 )
29
30     logger = logging.getLogger(__name__)
31
32     # A labeler turns (video_ref, guideline, sport) into rally intervals.
33     Labeler = Callable[[str, Optional[str], str], List[Interval]]
34
35
36     def _not_configured(video_ref: str, guideline: Optional[str], sport: str) ->
List[Interval]:
37         """Default labeler: refuses to run until a real model is injected.
38
39         Keeps `AutomatedProvider()` constructible (so the registry can key it as 'auto')
40         while making accidental use of an unconfigured oracle fail loudly rather than
41         silently returning no rallies.
42         """
43         raise NotImplementedError(
44             "AutomatedProvider has no labeler configured. Inject one via "
45             "annotation_registry.get('auto', labeler=<fn>, lineage='<model-family>')."
46             "Choosing the concrete oracle model is a tracked item ? see docs/BACKLOG.md
(GS-4)."
47         )
48
49
50     def warn_if_same_lineage(oracle_lineage: Optional[str], production_lineage:
Optional[str]) -> bool:
51         """Warn (and return True) if the oracle shares production's model lineage.
52
53         Encodes the oracle rule in code, not just docs: a same-lineage oracle can't be
54         trusted to catch production's failures. Callers wiring CI should pass both.
55         """
56         if oracle_lineage and production_lineage and oracle_lineage == production_lineage:
57             logger.warning(
58                 "Oracle lineage '%s' matches production's ? the regression test may be "
59                 "blind to shared failure modes. Use a different model lineage.",
60                 oracle_lineage,
61             )
62             return True
63         return False
64
65
66     class AutomatedProvider(AnnotationProvider):
67         def __init__(self, labeler: Optional[Labeler] = None, lineage: str = "unspecified"):
68             """
69             Args:
70                 labeler: callable producing rally intervals for a video. Defaults to a
71                         stub that raises until configured.
72                 lineage: the oracle's model family (e.g. 'gemini', 'qwen-vl') ? recorded
73                         so callers can enforce the oracle rule via warn_if_same_lineage.
74             """
75             self._labeler = labeler or _not_configured
76             self.lineage = lineage
77             # In-process synchronous job store: job_id -> labeled intervals.
78             self._jobs: Dict[str, List[Interval]] = {}
79
80         @property
81         def name(self) -> str:
82             return "auto"
83
84         def submit(self, video_ref: str, guideline: Optional[str] = None,
85                   sport: str = "badminton") -> str:
86             # Synchronous stub: run the labeler now and stash the result. A real async

```



```

87         # oracle (e.g. a queued API job) would kick off here and resolve in poll().
88         intervals = self._labeler(video_ref, guideline, sport)
89         intervals = sorted((float(s), float(e)) for s, e in intervals)
90         job_id = video_ref
91         self._jobs[job_id] = intervals
92         return job_id
93
94     def poll(self, job_id: str) -> str:
95         return DONE if job_id in self._jobs else FAILED
96
97     def fetch(self, job_id: str) -> List[Interval]:
98         if job_id not in self._jobs:
99             raise KeyError(f"AutomatedProvider: unknown job '{job_id}' (submit it
first)")
100         return self._jobs[job_id]
101
102
103     annotation_registry.register(AutomatedProvider)
104

```

16. user (2026-06-18T01:28:06.752Z)

```

1     """FileProvider ? ground-truth rally labels from pre-made CSV files.
2
3     This formalizes today's manual / ShuttleSet annotation flow behind the
4     `AnnotationProvider` interface: the "annotation job" is simply a CSV that already
5     exists on disk (the same `start,end` format `run_eval.py --scaffold` writes). The
6     job_id IS the resolved CSV path, so submit/poll/fetch all operate on that path.
7
8     It's the synchronous, zero-cost provider used as the default golden-set source and
9     as the stand-in in tests/CI before a human-vendor or automated provider is wired in.
10    """
11    import os
12    from typing import List, Optional
13
14    from backend.annotations.base import (
15        AnnotationProvider,
16        annotation_registry,
17        Interval,
18        DONE,
19        FAILED,
20    )
21    from backend.eval.annotations import load_annotations
22
23
24    class FileProvider(AnnotationProvider):
25        def __init__(self, csv_dir: Optional[str] = None, ext: str = ".csv"):
26            """
27            Args:
28                csv_dir: if set, a bare `video_ref` (not itself a path) is resolved to
29                        `

```

```

44         """
45         if os.path.isfile(video_ref):
46             return video_ref
47         if self.csv_dir:
48             return os.path.join(self.csv_dir, f"{video_ref}{self.ext}")
49         return video_ref
50
51     def submit(self, video_ref: str, guideline: Optional[str] = None,
52               sport: str = "badminton") -> str:
53         # Nothing to do ? the label file is expected to already exist. The job_id
54         # is the resolved CSV path. (guideline/sport are irrelevant for a file.)
55         return self._resolve(video_ref)
56
57     def poll(self, job_id: str) -> str:
58         return DONE if os.path.isfile(job_id) else FAILED
59
60     def fetch(self, job_id: str) -> List[Interval]:
61         if not os.path.isfile(job_id):
62             raise FileNotFoundError(f"annotation file not found: {job_id}")
63         return load_annotations(job_id)
64
65
66     annotation_registry.register(FileProvider)
67

```

17. assistant (2026-06-18T01:28:09.306Z)

Let me read the annotations base, reporting, and tools files.

18. user (2026-06-18T01:28:10.256Z)

```

1     """Annotation provider registry ? pluggable sources of ground-truth rally labels.
2
3     An `AnnotationProvider` turns a video reference into a list of `(start, end)` rally
4     intervals ? the same shape that `backend.eval.annotations.load_annotations`
5     produces and `backend.eval.metrics` consumes. That symmetry is the whole point:
6     golden-set ground truth can come from a CSV file, a human annotation vendor, or an
7     automated oracle behind one interface, so the regression flow
8     (`backend.eval.regression`) never needs to know where labels came from.
9
10    Mirrors the existing registry pattern in `backend/providers/base.py` and
11    `backend/indexer/base.py`. See `docs/GOLDEN_SET_VENDORING.md` (and ADR-006) for the
12    design rationale.
13    """
14    from abc import ABC, abstractmethod
15    from typing import Dict, List, Optional, Tuple, Type
16
17    # An (start, end) interval in seconds ? identical to backend.eval.metrics.Interval.
18    Interval = Tuple[float, float]
19
20    # Job lifecycle statuses returned by poll().
21    QUEUED = "queued"
22    RUNNING = "running"
23    DONE = "done"
24    FAILED = "failed"
25    STATUSES = (QUEUED, RUNNING, DONE, FAILED)
26
27
28    class AnnotationProvider(ABC):
29        """Abstract source of ground-truth rally intervals for a video.
30
31        The three-method contract (submit ? poll ? fetch) accommodates both
32        synchronous providers (a file already on disk) and asynchronous ones (a human
33        vendor that takes days). Synchronous providers can use the `annotate()`
34        convenience, which runs the full cycle in one call.

```

```

35     """
36
37     @property
38     @abstractmethod
39     def name(self) -> str:
40         """Unique registry key for this provider (e.g. 'file', 'vendor', 'auto')."""
41
42     @abstractmethod
43     def submit(self, video_ref: str, guideline: Optional[str] = None,
44                sport: str = "badminton") -> str:
45         """Submit a video for annotation; return an opaque job_id.
46
47         `video_ref` is provider-defined (a file path, a video_id/MD5, a URL).
48         `guideline` is the annotation spec (what counts as a rally start/end).
49         """
50
51     @abstractmethod
52     def poll(self, job_id: str) -> str:
53         """Return one of STATUSES for the given job."""
54
55     @abstractmethod
56     def fetch(self, job_id: str) -> List[Interval]:
57         """Return the completed annotations as sorted `(start, end)` intervals.
58
59         Should raise if the job is not DONE / the labels are unavailable, rather
60         than returning an empty list (which would silently skew downstream scores).
61         """
62
63     def annotate(self, video_ref: str, guideline: Optional[str] = None,
64                 sport: str = "badminton") -> List[Interval]:
65         """Convenience: submit + fetch for a provider that completes synchronously.
66
67         Async providers (e.g. a human vendor with a multi-day SLA) should be driven
68         via submit/poll/fetch explicitly instead ? calling this on one will raise
69         once the job isn't immediately DONE.
70         """
71         job_id = self.submit(video_ref, guideline=guideline, sport=sport)
72         status = self.poll(job_id)
73         if status != DONE:
74             raise RuntimeError(
75                 f"{self.name}.annotate(): job '{job_id}' is '{status}', not DONE. "
76                 f"Use submit/poll/fetch for asynchronous providers."
77             )
78         return self.fetch(job_id)
79
80
81 class AnnotationProviderRegistry:
82     """Registry of annotation providers, keyed by each provider's `name`."""
83
84     def __init__(self):
85         self._providers: Dict[str, Type[AnnotationProvider]] = {}
86
87     def register(self, provider_cls: Type[AnnotationProvider]) ->
88     Type[AnnotationProvider]:
89         """Register a provider class under its `name`. Returns the class (decorator-
90         friendly)."""
91         try:
92             key = provider_cls().name
93         except Exception:
94             # Providers needing constructor args fall back to the class name.
95             key = getattr(provider_cls, "__name__", "unknown")
96         self._providers[key] = provider_cls
97         return provider_cls
98
99     def get(self, name: str, **kwargs) -> AnnotationProvider:

```

```

98         """Instantiate the named provider, passing through any constructor kwargs."""
99         cls = self._providers.get(name)
100         if cls is None:
101             raise KeyError(
102                 f"Unknown annotation provider '{name}'. Available:
{self.list_available()}"
103             )
104         return cls(**kwargs)
105
106     def list_available(self) -> List[str]:
107         return list(self._providers.keys())
108
109
110 annotation_registry = AnnotationProviderRegistry()
111

```

19. user (2026-06-18T01:28:11.273Z)

```

1     """Build an observability report for one indexer run.
2
3     Pure-ish harvest: given the validation results/context, the persisted segments,
4     and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
5     *footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
6     so this module only records facts faithfully ? it does not decide warnings.
7
8     Imported only after `backend.reporting` confirms obsreport is installed, so it may
9     import obsreport freely.
10    """
11    from __future__ import annotations
12
13    import os
14    from pathlib import Path
15    from typing import Any, Dict, List, Optional
16
17    try:
18        import obsreport # soft dep; only needed for RunRecorder / load_spec at runtime
19    except ImportError:
20        obsreport = None # type: ignore[assignment]
21
22    _AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
23    _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
24
25
26    def load_spec() -> "obsreport.ReportSpec":
27        return obsreport.load_spec(_SPEC_PATH)
28
29
30    # --- media metadata -----
31    def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path:
Optional[str]) -> Dict[str, Any]:
32        """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
33        distinguishes a real probe from the absence of one (the fallback footgun)."""
34        meta: Dict[str, Any] = {"fps_source": "unknown"}
35        if video_path and os.path.exists(video_path):
36            try:
37                meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
38            except OSError:
39                pass
40
41        ffmeta = (val_context or {}).get("ffprobe_meta") or {}
42        streams = ffmeta.get("streams") or []
43        fmt = ffmeta.get("format") or {}
44        if streams:
45            s = streams[0]
46            w, h = s.get("width"), s.get("height")

```

```

47     meta["width"], meta["height"] = w, h
48     if w and h:
49         meta["resolution"] = f"{w}x{h}"
50     fps_str = s.get("r_frame_rate", "0/1")
51     try:
52         num, den = (int(x) for x in fps_str.split("/"))
53         if den:
54             meta["fps"] = round(num / den, 2)
55             meta["fps_source"] = "probed"
56     except (ValueError, ZeroDivisionError):
57         pass
58     if s.get("codec_name"):
59         meta["container"] = fmt.get("format_name") or s.get("codec_name")
60     if fmt.get("duration"):
61         try:
62             meta["duration_s"] = round(float(fmt["duration"]), 1)
63         except (ValueError, TypeError):
64             pass
65     if fmt.get("format_name"):
66         meta["container"] = fmt.get("format_name")
67
68     # Audio readiness signal (for Input Quality UX extension per review item 2).
69     # Guidelines make "record with audio on, mic unobstructed" a first-class requirement
70     # (ADR-007) because shuttle thwacks are a strong rally cue for recall.
71     # We only record presence here (cheap, from ffprobe already in context); the actual
72     # thwack onset probe (pipeline/audio/hit_detector) remains a diagnostic /
calibration
73     # tool for now.
74     audio_streams = [s for s in streams if (s or {}).get("codec_type") == "audio"]
75     meta["audio_present"] = len(audio_streams) > 0
76     return meta
77
78
79     # --- stage builders -----
80     def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
81         with rec.stage("validation") as st:
82             total = len(val_results)
83             passed = sum(1 for r in val_results if getattr(r, "passed", False))
84             st.add_metric("checks_passed", passed)
85             st.add_metric("checks_total", total)
86             failures = [r for r in val_results if not getattr(r, "passed", True)]
87             for r in failures:
88                 st.messages.append(f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r,
'message', '')}")
89             st.status = "error" if failures else ("ok" if total else "not_run")
90
91
92     def _segmentation_stage(
93         rec: "obsreport.RunRecorder",
94         play_segments: List[Dict[str, Any]],
95         candidates: List[Dict[str, Any]],
96         segmenter_requested: Optional[str],
97         segmenter_actual: Optional[str],
98         config_default: Optional[str],
99         was_reingest: bool,
100         ran: bool,
101     ) -> None:
102         with rec.stage("segmentation") as st:
103             if not ran:
104                 st.status = "not_run"
105             seg_count = len(play_segments)
106             total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
107             detector_tag = None
108             if play_segments:
109                 detector_tag = (play_segments[0].get("metadata") or {}).get("detector")

```

```

110         st.add_metric("candidate_windows", len(candidates))
111         st.add_metric("segment_count", seg_count, audience="both")
112         st.add_metric("total_play_s", total_play, unit="s")
113         # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
114         # pick it, and it differs from config's default (the genuine surprise case).
115         # An explicit --segmenter override or a halted run is not a divergence.
116         matches_default = True
117         if ran and segmenter_actual and config_default:
118             matches_default = (segmenter_actual == config_default)
119         st.details.update({
120             "segmenter_requested": segmenter_requested,
121             "segmenter_actual": segmenter_actual,
122             "config_default": config_default,
123             "segmenter_explicitly_requested": segmenter_requested is not None,
124             "matches_config_default": matches_default,
125             "was_reingest": was_reingest,
126             "detector": detector_tag,
127         })
128         # Surface metadata trustworthiness at the data level (not just in a warning):
129         # the motion baseline fabricates per-segment metadata; everything else measures
130         it.
131         if ran and segmenter_actual:
132             st.details["metadata_source"] = (
133                 "synthetic/heuristic ? NOT measured" if segmenter_actual == "motion"
134                 else "measured (detector/AI)"
135             )
136
137     def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
138                 play_segments: List[Dict[str, Any]], ran: bool,
139                 run_signals: Optional[Dict[str, Any]] = None) -> None:
140         """Only meaningful for AI-based segmenters. Records whether the provider key was
141         present ? the signal the silent_provider_noop rule keys on ? plus any in-process
142         run signals (backend.reporting.run_signals), e.g. oversize chunk refusals."""
143         ai_based = segmenter_actual in _AI_SEGMENTERS
144         provider = (config.get("ai_provider") or "gemini")
145         key_env = f"{provider.upper()}_API_KEY"
146         with rec.stage("ai_handoff") as st:
147             if not ran or not ai_based:
148                 st.status = "not_run" if not ran else "skipped"
149             st.details.update({
150                 "ai_based": ai_based,
151                 "provider": provider if ai_based else None,
152                 "model": config.get("ai_model") if ai_based else None,
153                 "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
154                 # Chunks the provider refused as over its upload cap (no analysis ran for
155                 # those time ranges). Detail name couples to the oversize_clip_skipped rule.
156                 "oversize_clips_skipped": int((run_signals or
157 {}).get("oversize_clips_skipped", 0)) if ai_based else None,
158             })
159             if ai_based:
160                 st.add_metric("confirmed_segments", len(play_segments))
161
162     def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
163                   video_meta: Dict[str, Any]) -> None:
164         total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
165         coverage = None
166         dur = video_meta.get("duration_s")
167         if dur:
168             coverage = round(100.0 * total / dur, 1)
169         notes = []
170         if not play_segments:
171             notes.append("No highlights were produced for this video.")

```

```

172         rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
173                             coverage_pct=coverage, notes=notes)
174
175
176     # --- top-level -----
177     def record_run(
178         *,
179         video_id: str,
180         video_path: Optional[str],
181         config: Dict[str, Any],
182         val_results: List[Any],
183         val_context: Optional[Dict[str, Any]],
184         play_segments: List[Dict[str, Any]],
185         candidates: List[Dict[str, Any]],
186         segmenter_requested: Optional[str] = None,
187         segmenter_actual: Optional[str] = None,
188         was_reingest: bool = False,
189         segmentation_ran: bool = True,
190         tool_version: Optional[str] = None,
191         spec: Optional["obsreport.ReportSpec"] = None,
192         run_signals: Optional[Dict[str, Any]] = None,
193         **recorder_kwargs: Any,
194     ) -> "obsreport.RunRecorder":
195         """Assemble a finalized recorder for an indexer run (does NOT write)."""
196         rec = obsreport.RunRecorder(
197             "badminton-highlight-indexer",
198             tool_version=tool_version,
199             video_id=video_id,
200             video_id_kind="file_md5",
201             video_path=video_path,
202             spec=spec if spec is not None else load_spec(),
203             **recorder_kwargs,
204         )
205         vm = video_meta_from_context(val_context, video_path)
206         # If we never probed an fps but segmentation still ran, the segmenter must have
207         # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,
208         # AND show the assumed numeric value so the report reads "30 (fallback)" not "?"
209         (fallback)".
210         if vm.get("fps_source") == "unknown" and segmentation_ran:
211             vm["fps_source"] = "fallback"
212             if vm.get("fps") is None:
213                 vm["fps"] = 30.0 # matches the runners' fallback (_probe_fps_width -> 30.0
214 / 1280)
215         rec.set_video_meta(**vm)
216
217         _validation_stage(rec, val_results)
218         config_default = (config.get("indexing") or {}).get("default_segmenter")
219         _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
220                             segmenter_actual, config_default, was_reingest,
221                             segmentation_ran)
222         _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran,
223                   run_signals)
224         _highlights(rec, play_segments, vm)
225
226         rec.finalize()
227         return rec
228
229

```

20. user (2026-06-18T01:28:11.494Z)

```

1     """In-process signal channel: pipeline internals -> the per-video report.
2
3     Some report-worthy facts (e.g. the provider refusing an oversize chunk) surface
4     deep inside a segmenter's worker threads, with no return path to the harvest ?
5     segmenters return only their segment list. Layers record those facts here keyed

```

```

6     by video_id; `emit_indexer_report` pops them when the run's report is built.
7
8     Process-local and best-effort BY DESIGN: recording must never break a run, the
9     store is consumed (popped) once per report, and signals for a video are simply
10    dropped if no report is emitted. No obsreport dependency ? importable everywhere.
11    """
12    from __future__ import annotations
13
14    import threading
15    from typing import Any, Dict
16
17    _lock = threading.Lock()
18    _signals: Dict[str, Dict[str, Any]] = {}
19
20
21    def incr(video_id: str, key: str, by: int = 1) -> None:
22        """Add `by` to the counter `key` for this video's in-flight run."""
23        if not video_id or by == 0:
24            return
25        with _lock:
26            sig = _signals.setdefault(video_id, {})
27            sig[key] = int(sig.get(key, 0)) + by
28
29
30    def pop(video_id: str) -> Dict[str, Any]:
31        """Consume (remove and return) all signals recorded for this video's run."""
32        with _lock:
33            return _signals.pop(video_id, {})
34

```

21. assistant (2026-06-18T01:28:16.817Z)

Now let me read the tools files.

22. user (2026-06-18T01:28:17.737Z)

```

1     """Slice individual rally clips out of a (long) annotated video.
2
3     Use cases:
4     - **Offline verification** ? pull rally N out of a 30-min match to eyeball its
5       boundaries against the labels.
6     - **Per-video calibration/training** ? grab a handful of labeled rallies as the
7       supervision the segmenter is tuned/adapted on for that specific video.
8
9     Given a labels CSV (the golden `rally_number,start_time,end_time[,start_mmss,
10    end_mmss]` shape) and a set of rally numbers, it writes one padded clip per rally:
11    <out_dir>/<video_stem>__rally_<n>.mp4
12
13    Padding matches the highlight compiler (begin 0.5 s / end 1.0 s) so a sliced clip
14    shows the same lead-in/out the model is trained and evaluated against. The window
15    math (`compute_clip_windows`) is pure and unit-tested; ffmpeg is only touched in
16    `slice_rallies`.
17    """
18    from __future__ import annotations
19
20    import argparse
21    import csv
22    import os
23    import subprocess
24    from dataclasses import dataclass
25    from typing import Dict, List, Optional, Tuple
26
27    from backend.eval.annotations import parse_timestamp
28
29    # Match VideoCompiler defaults so sliced clips frame a rally the same way.

```



```

30 BEGIN_PADDING = 0.5
31 END_PADDING = 1.0
32
33
34 @dataclass
35 class ClipWindow:
36     rally: str          # kept as a string to allow inserted ids like "7.5"
37     start: float        # padded, clamped
38     end: float          # padded, clamped
39
40
41 def _read_time(row: Dict[str, str], decimal_key: str, mmss_key: str) -> Optional[float]:
42     """Prefer the decimal-seconds column; fall back to mm:ss. None if unusable."""
43     for key in (decimal_key, mmss_key):
44         val = (row.get(key) or "").strip()
45         if val:
46             try:
47                 return parse_timestamp(val)
48             except ValueError:
49                 continue
50     return None
51
52
53 def load_rally_labels(csv_path: str) -> Dict[str, Tuple[float, float]]:
54     """Map rally_number -> (start, end) seconds, skipping rows with no usable times.
55
56     Accepts the golden schema (start_time/end_time decimal, optional start_mmss/
57     end_mmss). Rally numbers are kept as strings so inserted ids ("7.5") survive.
58     """
59     labels: Dict[str, Tuple[float, float]] = {}
60     with open(csv_path, newline="", encoding="utf-8") as f:
61         reader = csv.DictReader(f)
62         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
63 [])]
64         for row in reader:
65             row = {(k or "").strip().lower(): v for k, v in row.items()}
66             rally = (row.get("rally_number") or "").strip()
67             if not rally:
68                 continue
69             start = _read_time(row, "start_time", "start_mmss")
70             end = _read_time(row, "end_time", "end_mmss")
71             if start is None or end is None or end <= start:
72                 continue # missing/degenerate (e.g. a not-yet-filled MISSED row)
73             labels[rally] = (start, end)
74     return labels
75
76
77 def compute_clip_windows(labels: Dict[str, Tuple[float, float]],
78 rally_numbers: List[str],
79 begin_pad: float = BEGIN_PADDING,
80 end_pad: float = END_PADDING,
81 video_duration: Optional[float] = None) -> List[ClipWindow]:
82     """Apply padding and clamp to the video, for the requested rallies.
83
84     rally_numbers may be ["all"] to take every labeled rally (in time order).
85     Unknown rally ids are skipped (caller can diff to warn).
86     """
87     if len(rally_numbers) == 1 and rally_numbers[0].lower() == "all":
88         wanted = sorted(labels, key=lambda r: labels[r][0])
89     else:
90         wanted = [r for r in rally_numbers if r in labels]
91
92     windows: List[ClipWindow] = []
93     for rally in wanted:
94         start, end = labels[rally]

```

```

94         ps = max(0.0, start - begin_pad)
95         pe = end + end_pad
96         if video_duration is not None and video_duration > 0:
97             if ps >= video_duration:
98                 continue # entirely past the end
99             pe = min(pe, video_duration)
100         if pe <= ps:
101             continue
102         windows.append(ClipWindow(rally=rally, start=round(ps, 3), end=round(pe, 3)))
103     return windows
104
105
106 def _probe_duration(video_path: str) -> Optional[float]:
107     try:
108         out = subprocess.run(
109             ["ffprobe", "-v", "error", "-show_entries", "format=duration",
110              "-of", "default=noprint_wrappers=1:nokey=1", video_path],
111             capture_output=True, text=True, check=True,
112             ).stdout.strip()
113         return float(out)
114     except (subprocess.SubprocessError, OSError, ValueError):
115         return None
116
117
118 def slice_rallies(video_path: str, labels_csv: str, rally_numbers: List[str],
119                  out_dir: str, begin_pad: float = BEGIN_PADDING,
120                  end_pad: float = END_PADDING) -> List[str]:
121     """Write one padded clip per requested rally; returns the clip paths written."""
122     if not os.path.exists(video_path):
123         raise FileNotFoundError(f"video not found: {video_path}")
124     labels = load_rally_labels(labels_csv)
125     if not labels:
126         raise ValueError(f"no usable rally rows in {labels_csv}")
127     os.makedirs(out_dir, exist_ok=True)
128
129     duration = _probe_duration(video_path)
130     windows = compute_clip_windows(labels, rally_numbers, begin_pad, end_pad, duration)
131     if not windows:
132         raise ValueError("no rallies matched the request (check --rallies vs the CSV)")
133
134     stem = os.path.splitext(os.path.basename(video_path))[0]
135     written: List[str] = []
136     for w in windows:
137         safe_rally = w.rally.replace(".", "_")
138         out_path = os.path.join(out_dir, f"{stem}__rally_{safe_rally}.mp4")
139         cmd = [
140             "ffmpeg", "-y",
141             "-ss", str(w.start), "-to", str(w.end), "-i", video_path,
142             "-c:v", "libx264", "-preset", "superfast", "-crf", "22",
143             "-c:a", "aac", "-b:a", "128k",
144             out_path,
145         ]
146         try:
147             subprocess.run(cmd, capture_output=True, check=True)
148         except subprocess.CalledProcessError as e:
149             err = e.stderr.decode(errors="replace").strip()[-300:]
150             print(f"[slicer] rally {w.rally}: ffmpeg failed: {err}")
151             continue
152         if os.path.exists(out_path) and os.path.getsize(out_path) > 0:
153             written.append(out_path)
154             print(f"[slicer] rally {w.rally} -> {out_path}")
155     return written
156
157

```

```

158     def main() -> None:
159         p = argparse.ArgumentParser(description="Slice rally clips from an annotated
video.")
160         p.add_argument("--video", required=True, help="Source video file")
161         p.add_argument("--labels", required=True, help="Labels CSV (golden rally format)")
162         p.add_argument("--rallies", required=True,
163             help="Comma-separated rally numbers (e.g. 7,7.5,16) or 'all'")
164         p.add_argument("--out-dir", default="output/rally_clips", help="Output directory")
165         p.add_argument("--begin-pad", type=float, default=BEGIN_PADDING)
166         p.add_argument("--end-pad", type=float, default=END_PADDING)
167         args = p.parse_args()
168
169         rally_numbers = [r.strip() for r in args.rallies.split(",") if r.strip()]
170         written = slice_rallies(args.video, args.labels, rally_numbers,
171             args.out_dir, args.begin_pad, args.end_pad)
172         print(f"\nWrote {len(written)} clip(s) to {args.out_dir}")
173
174
175     if __name__ == "__main__":
176         main()
177

```

23. user (2026-06-18T01:28:18.212Z)

```

1     """Minimal Value Layer exports (first slice of review item 3).
2
3     Provides timeline exports (EDL, JSON, CSV) and basic stats over the
4     play_segments returned by the indexer (or query_play_segments). These are
5     the "time-saved editing is table stakes" deliverables and also give a
6     machine-readable foundation for later human correction / golden feeding
7     and clip galleries.
8
9     Kept deliberately small and pure (no DB, no side effects) so they are
10    easy to call from CLI, notebooks, or a future API endpoint.
11    """
12
13    from __future__ import annotations
14
15    import json
16    from typing import Any, Dict, Iterable, List, Optional
17
18
19    def _to_list(segments: Iterable[Dict[str, Any]]) -> List[Dict[str, Any]]:
20        return list(segments) if not isinstance(segments, list) else segments
21
22
23    def _segment_metadata(s: Dict[str, Any]) -> Dict[str, Any]:
24        """A play segment's metadata as a dict, whether stored inline (segmenter output)
25        or as a JSON string (DB round-trip via query_play_segments)."""
26        md = s.get("metadata") or {}
27        if isinstance(md, str):
28            try:
29                md = json.loads(md)
30            except Exception:
31                md = {}
32        return md if isinstance(md, dict) else {}
33
34
35    def _fmt_hms(seconds: float) -> str:
36        """HH:MM:SS, matching the segmenters' _fmt_ts so the summary block lines up with
37        the per-rally [rally] log lines and the existing timestamp style."""
38        total = max(0, int(float(seconds)))
39        h, rem = divmod(total, 3600)
40        m, s = divmod(rem, 60)
41        return f"{h:02d}:{m:02d}:{s:02d}"

```

```

42
43
44 def format_rally_summary(
45     segments: Iterable[Dict[str, Any]], title: str = "Rally summary"
46 ) -> str:
47     """Render a compact, scannable end-of-run block for manual QA against the rally CSV.
48
49     One header line (total rallies + total highlight duration in HH:MM:SS) followed by a
50     per-rally table ``# start end dur(s) shot_count`` in time order. Pure (no DB, no
I/O):
51     the same play_segment dicts that feed ``segments_to_csv`` / the rally CSV go in and
52     text comes out, so the log and CSV line up row-for-row for eyeball verification.
53
54     A segment with no ``shot_count`` in its metadata renders the count as ``-``.
55     """
56     segs = _to_list(segments)
57     total_dur = sum(
58         float(s.get("duration", float(s.get("end_time", 0)) - float(s.get("start_time",
0))))
59         for s in segs
60     )
61     header = (
62         f"=== {title}: {len(segs)} rallies, "
63         f"{_fmt_hms(total_dur)} ({total_dur:.1f}s) total play ==="
64     )
65     if not segs:
66         return header + "\n (no rallies detected)"
67
68     lines = [header, f"{'#':>3} {'start':<8} {'end':<8} {'dur(s)':>7} {'shots':>6}"]
69     for i, s in enumerate(segs, start=1):
70         start = float(s.get("start_time", 0.0))
71         end = float(s.get("end_time", start))
72         dur = float(s.get("duration", end - start))
73         shot = _segment_metadata(s).get("shot_count")
74         shot_str = "-" if shot is None else str(shot)
75         lines.append(
76             f"{i:>3} {_fmt_hms(start):<8} {_fmt_hms(end):<8} {dur:>7.1f}
{shot_str:>6}"
77         )
78     return "\n".join(lines)
79
80
81 def basic_stats(segments: Iterable[Dict[str, Any]]) -> Dict[str, Any]:
82     """Return simple aggregate stats for a list of play segments.
83
84     Keys: count, total_duration_s, avg_duration_s, min_duration_s, max_duration_s.
85     All durations are in seconds (float).
86     """
87     segs = _to_list(segments)
88     if not segs:
89         return {
90             "count": 0,
91             "total_duration_s": 0.0,
92             "avg_duration_s": 0.0,
93             "min_duration_s": 0.0,
94             "max_duration_s": 0.0,
95         }
96     durs = [float(s.get("duration", (s.get("end_time", 0) - s.get("start_time", 0))))
for s in segs]
97     total = sum(durs)
98     return {
99         "count": len(segs),
100         "total_duration_s": round(total, 2),
101         "avg_duration_s": round(total / len(durs), 2),
102         "min_duration_s": round(min(durs), 2),

```

```

103         "max_duration_s": round(max(durs), 2),
104     }
105
106
107 def segments_to_json(
108     segments: Iterable[Dict[str, Any]],
109     video_id: str,
110     video_path: Optional[str] = None,
111     meta: Optional[Dict[str, Any]] = None,
112 ) -> Dict[str, Any]:
113     """Return a JSON-serializable timeline dict.
114
115     Shape (stable):
116     {
117         "video_id": "...",
118         "video_path": "...",
119         "meta": {...},
120         "segments": [ {"id":, "start_time":, "end_time":, "duration":, "metadata": {...}},
... ],
121         "stats": basic_stats(...)
122     }
123     """
124     segs = _to_list(segments)
125     return {
126         "video_id": video_id,
127         "video_path": video_path,
128         "meta": meta or {},
129         "segments": segs,
130         "stats": basic_stats(segs),
131     }
132
133
134 def segments_to_csv(segments: Iterable[Dict[str, Any]], include_header: bool = True) ->
str:
135     """Return a CSV string (start,end,duration,shot_count,detector,...) .
136
137     Compatible with many editors and the existing golden/slicer schemas
138     (extra columns are harmless).
139     """
140     segs = _to_list(segments)
141     lines: List[str] = []
142     if include_header:
143         lines.append("start_time,end_time,duration,shot_count,shot_count_confidence,detector,id")
144     for s in segs:
145         md = s.get("metadata") or {}
146         shot = md.get("shot_count", "")
147         conf = md.get("shot_count_confidence", "")
148         det = md.get("detector", "")
149         lines.append(
150             f"{s.get('start_time',0)},{s.get('end_time',0)},{s.get('duration',0)},"
151             f"{shot},{conf},{det},{s.get('id','')}"
152         )
153     return "\n".join(lines) + "\n"
154
155
156 def segments_to_edl(segments: Iterable[Dict[str, Any]], title: str = "Highlights") ->
str:
157     """Return a simple EDL (CMX 3600-ish) for the segments.
158
159     Many NLEs will ingest this for a rough cut. Uses record timecode 00:00:00:00
160     as origin; clips are placed sequentially with the original source in/out.
161     """
162     segs = _to_list(segments)
163     lines: List[str] = [f"TITLE: {title}", "FCM: NON-DROP FRAME"]

```

```

164         for i, s in enumerate(segs, 1):
165             start = float(s.get("start_time", 0.0))
166             end = float(s.get("end_time", start))
167             # EDL is HH:MM:SS:FF (assume 30fps for the timecode display; real edit rate
168             # is irrelevant for a rough cut ? the numbers are just labels here).
169             def _tc(t: float) -> str:
170                 h = int(t // 3600)
171                 m = int((t % 3600) // 60)
172                 s_ = int(t % 60)
173                 f = int((t - int(t)) * 30) # 30 fps display
174                 return f"{h:02d}:{m:02d}:{s_:02d}:{f:02d}"
175             rec_in = _tc((i - 1) * 0.0) # place back-to-back in the EDL timeline
176             rec_out = _tc((i - 1) * 0.0 + (end - start))
177             src_in = _tc(start)
178             src_out = _tc(end)
179             lines.append(f"{i:03d}  AX  V      C          {rec_in} {rec_out} {src_in}
{src_out}")
180             lines.append(f"* FROM CLIP NAME: {title} {i}")
181             lines.append("")
182         return "\n".join(lines)
183

```

24. assistant (2026-06-18T01:28:21.391Z)

Now let me look at the `\_\_init\_\_.py` files and the execution_policy module that the Gemini provider depends on, to verify contracts.

25. user (2026-06-18T01:28:22.373Z)

```

1     from typing import Dict, Type
2     from backend.providers.base import AIVideoProvider
3     from backend.providers.gemini import GeminiProvider
4
5     class ProviderRegistry:
6         def __init__(self):
7             self._registry: Dict[str, Type[AIVideoProvider]] = {}
8             # Register Gemini provider
9             self.register("gemini", GeminiProvider)
10
11         def register(self, name: str, provider_cls: Type[AIVideoProvider]):
12             self._registry[name] = provider_cls
13
14         def get(self, name: str, api_key: str, model_name: str) -> AIVideoProvider:
15             if name not in self._registry:
16                 raise KeyError(f"AI Provider '{name}' is not registered in the
ProviderRegistry. Available: {list(self._registry.keys())}")
17             return self._registry[name](api_key=api_key, model_name=model_name)
18
19         def list_available(self) -> Dict[str, str]:
20             return {name: cls.__doc__ or f"{name.capitalize()} provider" for name, cls in
self._registry.items()}
21
22         provider_registry = ProviderRegistry()
23

```

26. user (2026-06-18T01:28:23.307Z)

```

1     """Per-video observability reports for the indexer.
2
3     `obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
4     no-op and the pipeline runs exactly as before. The single entrypoint
5     `emit_indexer_report(...)` is designed to be called from a `finally` block and
6     NEVER raises ? emitting a report must not be able to break a processing run.
7     """

```

```

8     from __future__ import annotations
9
10    from typing import Any, Dict, List, Optional
11
12    from . import run_signals # no obsreport dependency ? always importable
13
14    try: # soft dependency ? never hard-fail the pipeline on a reporting import
15        import obsreport # noqa: F401
16        from . import harvest
17
18        OBSREPORT_AVAILABLE = True
19    except Exception: # pragma: no cover - exercised only when obsreport is absent
20        OBSREPORT_AVAILABLE = False
21
22
23    def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
24        return bool((config or {}).get("report", {}).get("enabled", True))
25
26
27    def emit_indexer_report(
28        *,
29        db,
30        video_id: str,
31        video_path: Optional[str],
32        config: Optional[Dict[str, Any]],
33        val_results: List[Any],
34        val_context: Optional[Dict[str, Any]] = None,
35        segmenter_requested: Optional[str] = None,
36        segmenter_actual: Optional[str] = None,
37        was_reingest: bool = False,
38        segmentation_ran: bool = True,
39    ) -> Optional[Dict[str, str]]:
40        """Harvest + write the report. Returns the written file paths, or None if
41        reporting is unavailable/disabled/failed. Never raises."""
42        config = config or {}
43        # Accept either a plain dict or a typed config object (e.g.
44        backend.config.AppConfig).
45        # Normalize to a plain nested dict so harvest's dict access works uniformly.
46        if hasattr(config, "model_dump"):
47            try:
48                config = config.model_dump(mode="json")
49            except Exception:
50                config = {}
51        # Consume run signals unconditionally (even when reporting is off) so the
52        # process-local store never accumulates entries for finished runs.
53        signals = run_signals.pop(video_id)
54        if not OBSREPORT_AVAILABLE or not report_enabled(config):
55            return None
56        try:
57            play_segments = db.query_play_segments(video_id, {}) if db is not None else []
58            candidates = db.query_candidate_segments(video_id) if db is not None else []
59            rec = harvest.record_run(
60                video_id=video_id,
61                video_path=video_path,
62                config=config,
63                val_results=val_results,
64                val_context=val_context,
65                play_segments=play_segments,
66                candidates=candidates,
67                segmenter_requested=segmenter_requested,
68                segmenter_actual=segmenter_actual,
69                was_reingest=was_reingest,
70                segmentation_ran=segmentation_ran,
71                run_signals=signals,
72            )

```

```

72         output_dir = (config.get("report") or {}).get("output_dir")
73         res = rec.write(output_dir=output_dir)
74         return res.paths or None
75     except Exception: # pragma: no cover - defensive; reporting must never break a run
76         return None
77

```

27. user (2026-06-18T01:28:23.834Z)

```

1     """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3     Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4     - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5     for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6     so it is abandoned, not joined ? it never blocks the process.
7     - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8     ``max_wait_sec`` deadline).
9
10    **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11    waiting (a fake clock advances instantly).
12
13    **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14    transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15    ``execution.policy`` logger to watch the poll trail.
16    """
17    from __future__ import annotations
18
19    import logging
20    import threading
21    import time as _time
22    from dataclasses import asdict, dataclass
23    from enum import Enum
24    from typing import Callable, Optional, TypeVar
25
26    T = TypeVar("T")
27
28    #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
29    LOG = logging.getLogger("execution.policy")
30
31
32    class WaitMode(str, Enum):
33        """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
34        WAIT_AND_RESUME = "wait_and_resume" # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
35        ABORT = "abort" # one attempt; on hang/fail, stop immediately
36        BLOCK = "block" # bounded in-process blocking wait (no
reschedule)
37
38
39    class PolicyTimeout(TimeoutError):
40        """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
41
42
43    @dataclass(frozen=True)
44    class ExecutionPolicy:

```



```

45     """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
46     mode: WaitMode = WaitMode.WAIT_AND_RESUME
47     poll_every_n_sec: float = 10.0
48     op_timeout_sec: float = 120.0          # HARD per-attempt deadline ? the hang-killer
that was missing
49     max_wait_sec: float = 1800.0          # total wall budget for a poll loop
50     max_retries: int = 5                  # retries (=> max_retries+1 attempts) for
failures/hangs
51     backoff_base_sec: float = 3.0         # exponential: backoff_base * 2**attempt
52     on_timeout: str = "reschedule"        # reschedule | partial | fail (consumed by
P2/P3)
53     resumable: bool = True
54
55     def to_dict(self) -> dict:
56         d = asdict(self)
57         d["mode"] = self.mode.value
58         return d
59
60     @classmethod
61     def from_dict(cls, d: dict) -> "ExecutionPolicy":
62         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
63         kw = {k: v for k, v in d.items() if k in known}
64         if "mode" in kw and not isinstance(kw["mode"], WaitMode):
65             kw["mode"] = WaitMode(kw["mode"])
66         return cls(**kw)
67
68
69 #: The sane default (also what a job gets if it declares no policy).
70 DEFAULT_POLICY = ExecutionPolicy()
71
72
73 def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
74     """Deterministic exponential backoff (no jitter ? reproducible tests)."""
75     return policy.backoff_base_sec * (2 ** attempt)
76
77
78 def run_bounded(fn: Callable[[], T], policy: ExecutionPolicy = DEFAULT_POLICY, *,
79               label: str = "op", logger: Optional[logging.Logger] = None,
80               sleep: Callable[[float], None] = _time.sleep) -> T:
81     """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.
82
83     Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
84     last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
85     """
86     log = logger or LOG
87     attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
88     last_exc: Optional[BaseException] = None
89     for attempt in range(attempts):
90         box: dict = {}
91         done = threading.Event()
92
93         # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
94         # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
95         def _runner(_box: dict = box, _done: threading.Event = done) -> None:
96             try:
97                 _box["val"] = fn()
98             except BaseException as e:          # relay ANY error (incl. timeouts) to the
caller thread
99                 _box["exc"] = e

```

```

100         finally:
101             _done.set()
102
103         threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
104         if done.wait(timeout=policy.op_timeout_sec):
105             if "exc" in box:
106                 last_exc = box["exc"]
107                 log.info("%s: attempt %d/%d failed: %s", label, attempt + 1, attempts,
last_exc)
108             else:
109                 if attempt:
110                     log.info("%s: succeeded on attempt %d/%d", label, attempt + 1,
attempts)
111                     return box["val"] # type: ignore[return-value]
112             else:
113                 last_exc = PolicyTimeout(f"{label} hung >
op_timeout_sec={policy.op_timeout_sec}s")
114                 log.warning("%s: attempt %d/%d HUNG (>%.0fs) ? abandoning", label, attempt +
1, attempts,
115                             policy.op_timeout_sec)
116                 if attempt < attempts - 1:
117                     wait = _backoff_sec(policy, attempt)
118                     log.debug("%s: backing off %.1fs before retry", label, wait)
119                     sleep(wait)
120                 assert last_exc is not None
121                 log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
122                 raise last_exc
123
124
125     def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy =
DEFAULT_POLICY, *,
126                  label: str = "op", logger: Optional[logging.Logger] = None,
127                  sleep: Callable[[float], None] = _time.sleep,
128                  monotonic: Callable[[], float] = _time.monotonic) -> T:
129         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
130         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
131
132         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
133         """
134         log = logger or LOG
135         start = monotonic()
136         polls = 0
137         log.info("%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
138                 label, policy.poll_every_n_sec, policy.max_wait_sec)
139         while True:
140             result = check()
141             if result is not None:
142                 log.info("%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() -
start)
143                 return result
144             elapsed = monotonic() - start
145             if elapsed >= policy.max_wait_sec:
146                 log.warning("%s: still not ready after %.0fs (max_wait) ? giving up", label,
elapsed)
147                 raise PolicyTimeout(f"{label} not ready within
max_wait_sec={policy.max_wait_sec}s")
148             polls += 1
149             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
150             sleep(policy.poll_every_n_sec)
151

```

Now I have a good understanding. Let me trace some potential issues carefully.

In `gemini.py`, the `poll\_until` `\_processed` closure returns `None` while `state.name == "PROCESSING"`, otherwise returns `video\_file`. But there is a subtle issue: `poll\_until` raises `PolicyTimeout` if `check()` keeps returning None. The code catches `PolicyTimeout` separately from `Exception`. Let me check the FAILED state handling more carefully.

When `video\_file.state.name == "FAILED"`, `\_processed()` returns `video\_file` (not None), so `poll\_until` returns it. Then line 100 checks `if video\_file.state.name == "FAILED"`. That's fine.

Let me look closer at the `\_segment\_metadata` / metadata handling and the relevance config consumers, plus check how callers use `analyze\_video`'s return. Let me look for callers and tests.

29. user (2026-06-18T01:28:32.466Z)

```
config.json:12:         "court_green_lower": [
config.json:22:         "court_pixels_min_ratio": 0.05
README.md:165:         "court_green_lower": [35, 40, 40],
README.md:167:         "court_pixels_min_ratio": 0.05
README.md:228:Our default relevance check scans for the typical indoor green badminton court
floor color space (HSV). If your court is blue, or uses standard wood grain flooring, simply
adjust the HSV values inside your `config.json` under `validation.relevance.court_green_lower`
and `court_green_upper` to match your court, or lower the `court_pixels_min_ratio` to `0.0` to
bypass.
docs\CODE_MAP.md:130:[Omitted long matching line]
docs\CODE_MAP.md:208:[Omitted long matching line]
backend\config\models.py:19:     court_green_lower: list[int] = Field(default=[35, 40, 40])
backend\config\models.py:21:     court_pixels_min_ratio: float = Field(default=0.05, ge=0.0,
le=1.0)
tests\test_config.py:145:         "validation": {"relevance": {"court_pixels_min_ratio":
0.1}},
tests\test_config.py:152:         assert cfg.validation.relevance.court_pixels_min_ratio == 0.1
tests\test_config.py:275:     assert validation["relevance"].get("court_pixels_min_ratio") ==
0.05
backend\sports\base.py:16:     def get_relevance_config(self) -> dict:
backend\pipeline\validators\relevance.py:44:         sport_rel_cfg =
sport_profile.get_relevance_config()
backend\pipeline\validators\relevance.py:52:         hsv_lower =
np.array(rel_cfg.get("court_green_lower", sport_rel_cfg.get("court_green_lower", [35, 40, 40])))
backend\pipeline\validators\relevance.py:54:         min_court_ratio =
rel_cfg.get("court_pixels_min_ratio", sport_rel_cfg.get("court_pixels_min_ratio", 0.05))
backend\sports\badminton.py:65:     def get_relevance_config(self) -> dict:
backend\sports\badminton.py:67:         "court_green_lower": [35, 40, 40],
backend\sports\badminton.py:69:         "court_pixels_min_ratio": 0.05
tests\test_sport_profiles.py:21:     relevance_cfg = profile.get_relevance_config()
tests\test_sport_profiles.py:22:     assert "court_green_lower" in relevance_cfg
tests\test_sport_profiles.py:24:     assert "court_pixels_min_ratio" in relevance_cfg
```

30. user (2026-06-18T01:28:33.098Z)

```
backend\eval\gemini_refine.py:87:         raw, _ = provider.analyze_video(clip, prompt,
chunk_duration=ce - cs, label=f"cant{idx}")
backend\eval\gemini_refine.py:176:         raw, _ = provider.analyze_video(clip, prompt,
chunk_duration=cend - cstart, label=f"chunk{ci}")
backend\providers\base.py:10:     def analyze_video(self, video_path: str, prompt: str,
backend\providers\gemini.py:38:     def analyze_video(self, video_path: str, prompt: str,
tests\test_gemini.py:26:     mock_provider.analyze_video.return_value = ([
tests\test_gemini.py:61:     def analyze_video_side_effect(path, prompt, chunk_duration, label,
**kwargs):
tests\test_gemini.py:67:     mock_provider.analyze_video.side_effect = analyze_video_side_effect
tests\test_gemini.py:101:     mock_provider.analyze_video.return_value = analyze_return or (
tests\test_gemini.py:178:     mock_provider.analyze_video.return_value = (
tests\test_fusion_hybrid.py:42:     provider.analyze_video.return_value = (provider_segments or
```

```

[{"start_time": 0.5, "end_time": 2.0}], {}))
tests\test_fusion_hybrid.py:87:    assert provider.analyze_video.call_count == 2
tests\test_fusion_hybrid.py:95:    assert provider.analyze_video.call_count == 1
tests\test_fusion_hybrid.py:105:   assert provider.analyze_video.call_count == 2
tests\test_fusion_hybrid.py:146:   assert provider.analyze_video.call_count == 1           # only
window A (>=2 players) handed off
backend\pipeline\segmenters\yolo_hybrid.py:291:           segments, metrics =
provider.analyze_video(w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi+1}")
tests\test_provider_gemini.py:27:           segments, metrics = provider.analyze_video("dummy.mp4",
"dummy prompt", chunk_duration=10.0)
tests\test_provider_gemini.py:52:           segments, metrics = provider.analyze_video("dummy.mp4",
"dummy prompt")
tests\test_provider_gemini.py:64:           segments, metrics =
provider.analyze_video("big.mp4", "p")
tests\test_provider_gemini.py:80:           segments, metrics = provider.analyze_video("dummy.mp4",
"dummy prompt")
backend\pipeline\segmenters\trajectory_hybrid.py:285:           segments, _ =
provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
docs\EXECUTION_POLICY.md:13: `backend/providers/gemini.py::analyze_video` has **two unbounded
waits**:
docs\EXECUTION_POLICY.md:83: `op_timeout_sec` + retry/backoff; `poll_until` = bounded readiness
poll). `GeminiProvider.analyze_video`
docs\CODE_MAP.md:73: -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt) ? provider_registry
docs\CODE_MAP.md:209:[Omitted long matching line]
docs\CODE_MAP.md:287:- **Gemini free-tier / API-key dead path:** hybrids + gemini return ``[]`
(not error) if `{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op.
`analyze_video` also returns ``[]` on real failure -> empty != "no rallies". The reporting
`silent_provider_noop`/`ai_empty_vs_no_rallies` rules exist precisely to disambiguate this in
the report.
backend\pipeline\segmenters\gemini.py:178:           chunk_segments, metrics =
provider.analyze_video(
backend\pipeline\segmenters\gemini.py:278:           ai_segments, metrics =
provider.analyze_video(
docs\BACKLOG.md:91:- ? **Item 7 (unify hybrid segmenters) should land before the report's real
AI-provider timing.** The report records AI timing as `not_captured`; once
`wasb_hybrid`/`tracknet_hybrid` are unified there is ONE place to thread a recorder callback
through `provider.analyze_video` ? the report. **Revisit:** with item 7 (Claude to add the
timing hook there).
tests\test_tracknet_hybrid.py:54:   provider.analyze_video.return_value = (
tests\test_tracknet_hybrid.py:122:   provider.analyze_video.return_value = ([{"start_time":
0.5, "end_time": 3.0}], {})
tests\test_tracknet_hybrid.py:144:   provider.analyze_video.return_value = (
tests\test_trajectory_hybrid_equivalence.py:43:   provider.analyze_video.return_value =
(provider_segments, {})
tests\test_yolo_hybrid.py:64:   mock_provider.analyze_video.return_value = ([
tests\test_yolo_hybrid.py:131:   mock_provider.analyze_video.return_value = ([
tests\test_yolo_hybrid.py:200:   mock_provider.analyze_video.return_value = ([
tests\test_yolo_hybrid.py:251:   mock_provider.analyze_video.return_value = ([

```

31. assistant (2026-06-18T01:28:36.535Z)

Let me look at the gemini segmenter to understand how the returned segments are consumed - particularly whether the dicts are accessed in a way that assumes keys present. Also let me check `gemini\_refine.py` which is in eval but consumes the provider output.

32. user (2026-06-18T01:28:38.153Z)

```

150
151         def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
152             chunk_end = min(chunk_start + chunk_duration, video_duration)
153             actual_chunk_dur = chunk_end - chunk_start
154             chunk_label = f"Chunk {ci+1}/{num_chunks}"
155             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")

```

```

156
157         # Extract chunk with ffmpeg
158         logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
159         success = extract_video_chunk(
160             video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
161             reencode=ai_scale_width > 0,
162             scale_width=ai_scale_width if ai_scale_width > 0 else None,
163         )
164         if not success:
165             logger.error(f"Failed to extract {chunk_label}. Skipping.")
166             return chunk_label, [], {}
167
168         # Measure chunk size
169         try:
170             chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
171         except Exception:
172             chunk_size_mb = 0.0
173
174         # Get prompt from sport profile
175         prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
176
177         # Process this chunk through the provider
178         chunk_segments, metrics = provider.analyze_video(
179             chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
180         )
181         metrics["file_size_mb"] = chunk_size_mb
182         metrics["segments_found"] = len(chunk_segments)
183
184         # Offset timestamps back to full-video coordinates
185         for segment in chunk_segments:
186             if "start_time" in segment:
187                 try:
188                     segment["start_time"] = float(segment["start_time"]) +
chunk_start
189                 except (ValueError, TypeError):
190                     pass
191             if "end_time" in segment:
192                 try:
193                     segment["end_time"] = float(segment["end_time"]) +
chunk_start
194                 except (ValueError, TypeError):
195                     pass
196
197         logger.info(f"{chunk_label} returned {len(chunk_segments)} play
segments.")
198
199         # Clean up chunk file
200         try:
201             os.remove(chunk_path)
202         except Exception:
203             pass
204
205         return chunk_label, chunk_segments, metrics
206
207     # Run chunks in parallel. Workers come from indexing.ai_max_workers (same
knob
208     # the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
209     # gently ramp a cold/prepaid Gemini project, whose burst throttling returns
a
210     # misleading "credits depleted" 429 under 5 simultaneous uploads.
211     all_metrics = {}
212     max_workers = max(1, int(self.config.get("indexing",
{})).get("ai_max_workers", 5)))

```

```

213         logger.info("Starting parallel processing with up to %d concurrent
workers...", max_workers)
214         t_chunking_start = time.time()
215         with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as
executor:
216             futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
217             for future in concurrent.futures.as_completed(futures):
218                 label, segments, metrics = future.result()
219                 ai_segments.extend(segments)
220                 if metrics:
221                     all_metrics[label] = metrics
222
223             # Oversize refusals must be LOUD: each one is a time range with NO analysis,
224             # so "0 segments / success" here would be a silent failure, not "no
rallies".
225             oversize_skips = sum(1 for m in all_metrics.values() if
m.get("oversize_skipped"))
226             if oversize_skips:
227                 logger.error(
228                     f"{oversize_skips}/{num_chunks} chunks exceeded the provider upload
cap and were "
229                     f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time
ranges. "
230                     f"Set indexing.ai_chunk_scale_width (default 1280) so chunks are re-
encoded/"
231                     f"downscaled before upload."
232                 )
233             run_signals.incr(video_id, "oversize_clips_skipped", oversize_skips)
234
235             # Print Summary Log
236             summary_lines = [
237                 "",
238                 "-" * 60,
239                 "                GEMINI CHUNKING SUMMARY",
240                 "-" * 60,
241                 f" Total time: {time.time() - t_chunking_start:.1f}s",
242                 f" Total chunks: {num_chunks}",
243                 f" Total play segments found: {len(ai_segments)}",
244                 "-" * 60,
245                 f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments':<8}",
246                 "-" * 60,
247             ]
248             for i in range(num_chunks):
249                 lbl = f"Chunk {i+1}/{num_chunks}"
250                 m = all_metrics.get(lbl, {})
251                 sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
252                 upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
253                 prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
254                 gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
255                 seg = m.get('segments_found', 0) if m else 0
256                 summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} |
{gen:<8} | {seg:<8}")
257             summary_lines.append("-" * 60)
258             logger.info("\n".join(summary_lines))
259
260             # Clean up local temp files
261             cleanup_temp_files()
262             try:
263                 os.rmdir(chunks_dir)
264             except Exception:
265                 pass
266
267             logger.info(f"All chunks processed. {len(ai_segments)} total play segments

```

```

before deduplication.")
268
269         else:
270             # --- SINGLE-FILE PROCESSING (short videos) ---
271             try:
272                 file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
273             except Exception:
274                 file_sz = 0.0
275
276             # Get prompt from sport profile
277             prompt = sport_profile.get_prompt(chunk_duration=video_duration)
278             ai_segments, metrics = provider.analyze_video(
279                 video_to_upload, prompt, chunk_duration=video_duration,
label="SingleFile"
280             )
281             if metrics.get("oversize_skipped"):
282                 logger.error(
283                     f"The video ({file_sz:.0f}MB) exceeded the provider upload cap and
was "
284                     f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no
rallies'."
285                 )
286                 run_signals.incr(video_id, "oversize_clips_skipped", 1)
287             summary_lines = [
288                 "",
289                 "=" * 50,
290                 "                GEMINI SINGLE FILE SUMMARY",
291                 "=" * 50,
292                 f" File Size: {file_sz:.1f} MB",
293                 f" Upload time: {metrics.get('upload_time', 0):.1f}s",
294                 f" API Processing time: {metrics.get('process_time', 0):.1f}s",
295                 f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
296                 f" Play segments found: {len(ai_segments)}",
297                 "=" * 50,
298             ]
299             logger.info("\n".join(summary_lines))
300             cleanup_temp_files()
301
302             if not ai_segments:
303                 return []
304
305             # Populate SQLite DB with parsed play segments
306             segments_data = []
307             logger.info(f"Processing {len(ai_segments)} play segments through
validation...")
308
309             def parse_time(val) -> float:

```

33. assistant (2026-06-18T01:28:47.922Z)

Note: the gemini segmenter consumes `oversize\_clips\_skipped` from run_signals (line 233/286), but the reporting harvest in `harvest.py` line 156 reads `run\_signals.get("oversize\_clips\_skipped", 0)`. These match.

Now let me look at the harvest report key. In `\_ai\_stage` it reads `oversize\_clips\_skipped`, but in `gemini.py` provider's `analyze\_video` it sets `metrics["oversize\_skipped"]`. The segmenter converts the per-chunk `oversize\_skipped` into the `oversize\_clips\_skipped` signal. Consistent.

Let me now focus on the `harvest.py` `video\_meta\_from\_context` ? the `r\_frame\_rate` parsing. The split on `/`:

```

```python
num, den = (int(x) for x in fps_str.split("/"))
```

```

If ``r_frame_rate`` is something like ``"30000/1001"`` that's fine. But what if it's a malformed value with more than 2 parts or just ``"30"`` (no slash)? ``"30".split("/")`` ? ``["30"]``, then ``num, den = (int(x) for x in [...])`` would raise ``ValueError: not enough values to unpack``, which is caught by the ``except (ValueError, ZeroDivisionError)``. Good. If ``"30/1/2"`` ? too many values ? ``ValueError`` ? caught. OK.

Let me now examine the ``export.py`` ``_tc`` function for negative time and the EDL more carefully, and verify the ``format_rally_summary`` ``total_dur`` fallback. Let me check the tests for export and look at ``_fmt_hms`` with negative values.

Let me look more carefully at a subtle issue in ``automated_provider.py``. The ``submit`` does ``sorted((float(s), float(e)) for s, e in intervals)``. But ``poll`` returns DONE if ``job_id`` in ``self._jobs``. The ``job_id = video_ref``. If two different videos share a ref... unlikely. Let me check the test for ``automated_provider``.

34. user (2026-06-18T01:28:48.646Z)

tests\test_annotation_providers.py

35. user (2026-06-18T01:28:49.254Z)

tests\test_annotation_providers.py
tests\test_automated_provider.py
tests\test_provider_gemini.py
tests\test_providers.py

36. user (2026-06-18T01:28:49.591Z)

tests\test_export_wasb_segments.py

37. user (2026-06-18T01:28:50.095Z)

tests\test_rally_slicer.py

38. user (2026-06-18T01:28:50.285Z)

tests\test_reporting_harvest.py

39. assistant (2026-06-18T01:28:56.890Z)

Let me look at the ``harvest.py`` more closely on the ``container`` logic, and also check ``_read_time`` / ``compute_clip_windows`` for the slicer against its tests. But first let me focus on a specific suspicion in ``harvest.py``: the ``container`` assignment.

```
In `harvest.py`, lines 58-59:
```python
if s.get("codec_name"):
 meta["container"] = fmt.get("format_name") or s.get("codec_name")
...

then lines 65-66:
```python
if fmt.get("format_name"):
    meta["container"] = fmt.get("format_name")
...

```

This is redundant but not a bug.

Let me examine the ``r_frame_rate`` more ? actually the real concern. Look at line 55: ``fps_str = s.get("r_frame_rate", "0/1")``. If ``r_frame_rate`` key exists but value is ``None`` (ffprobe sometimes emits null), then ``s.get("r_frame_rate", "0/1")`` returns ``None``, and ``None.split("/")`` ? ``AttributeError``, which is NOT caught by ``except (ValueError, ZeroDivisionError)``. Let me verify this is reachable and check the test.

40. user (2026-06-18T01:28:57.839Z)


```

1      """Tests for the indexer observability harvest + footgun rules.
2
3      These exercise the broken-run scenarios the report is meant to make debuggable.
4      obsreport is a soft dependency; skip cleanly if it isn't installed.
5      """
6      import os
7
8      import pytest
9
10     pytest.importorskip("obsreport")
11
12     from backend.reporting import emit_indexer_report, harvest, run_signals # noqa: E402
13     from backend.pipeline.validators.base import ValidationResult # noqa: E402
14
15
16     class _FakeDB:
17         def __init__(self, play=None, candidates=None, existing=False):
18             self._play = play or []
19             self._candidates = candidates or []
20             self._existing = existing
21
22         def query_play_segments(self, video_id, filters):
23             return list(self._play)
24
25         def query_candidate_segments(self, video_id, detector=None, only_unlabeled=False):
26             return list(self._candidates)
27
28         def get_video(self, video_id):
29             return {"id": video_id} if self._existing else None
30
31
32     def _vr(name, passed, message="ok"):
33         return ValidationResult(validator_name=name, passed=passed, message=message)
34
35
36     _FFPROBE_OK = {
37         "streams": [{"width": 1920, "height": 1080, "r_frame_rate": "60000/1001",
38 "codec_name": "hevc"}],
39         "format": {"format_name": "mov,mp4,m4a", "duration": "676.0", "size": "5500000000"},
40     }
41
42     def _record(**kw):
43         base = dict(
44             video_id="md5abc",
45             video_path=None,
46             config={"ai_provider": "gemini", "ai_model": "gemini-flash", "indexing":
{"default_segmenter": "wasb_hybrid"}},
47             val_results=[_vr("format_check", True), _vr("relevance_check", True)],
48             val_context={"ffprobe_meta": _FFPROBE_OK},
49             play_segments=[{"duration": 5.0, "metadata": {"detector": "wasb-hybrid-
gemini"}}],
50             candidates=[{"id": 1}, {"id": 2}],
51             segmenter_requested="wasb_hybrid",
52             segmenter_actual="wasb_hybrid",
53             was_reingest=False,
54             segmentation_ran=True,
55         )
56         base.update(kw)
57         return harvest.record_run(**base).envelope
58
59
60     def _codes(env):
61         return {f.code for f in env.flags}
62

```

```

63
64 def test_healthy_run_is_clean(monkeypatch):
65     monkeypatch.setenv("GEMINI_API_KEY", "x")
66     env = _record()
67     assert env.verdict == "pass"
68     assert _codes(env) == set()
69     assert env.video_meta.fps == 59.94
70     assert env.video_meta.fps_source == "probed"
71     seg = env.get_stage("segmentation")
72     assert {m.key: m.value for m in seg.metrics}["segment_count"] == 1
73     assert {m.key: m.value for m in seg.metrics}["candidate_windows"] == 2
74     assert env.lint() == []
75
76
77 def test_silent_provider_noop_fires_when_no_key_and_zero_segments(monkeypatch):
78     monkeypatch.delenv("GEMINI_API_KEY", raising=False)
79     env = _record(play_segments=[])
80     assert "silent_provider_noop" in _codes(env)
81     assert env.verdict == "fail"
82     flag = next(f for f in env.flags if f.code == "silent_provider_noop")
83     assert flag.faq_ref == "README#Q7"
84
85
86 def test_ai_empty_warns_when_key_present_but_zero_segments(monkeypatch):
87     monkeypatch.setenv("GEMINI_API_KEY", "x")
88     env = _record(play_segments=[])
89     codes = _codes(env)
90     assert "ai_empty_vs_no_rallies" in codes
91     assert "silent_provider_noop" not in codes
92     assert env.verdict == "pass_with_warnings"
93
94
95 def test_oversize_clip_skip_flags_error(monkeypatch):
96     """The provider refusing oversize chunks (4K stream-copy) must surface as an ERROR
97     flag ? the observed live failure was all 7 chunks skipped + '0 segments /
98     success'."""
99     monkeypatch.setenv("GEMINI_API_KEY", "x")
100     env = _record(play_segments=[], run_signals={"oversize_clips_skipped": 7})
101     assert "oversize_clip_skipped" in _codes(env)
102     assert env.verdict == "fail"
103     assert env.get_stage("ai_handoff").details["oversize_clips_skipped"] == 7
104
105 def test_no_oversize_signal_records_zero_and_stays_clean(monkeypatch):
106     monkeypatch.setenv("GEMINI_API_KEY", "x")
107     env = _record() # no run_signals at all
108     assert env.get_stage("ai_handoff").details["oversize_clips_skipped"] == 0
109     assert "oversize_clip_skipped" not in _codes(env)
110
111
112 def test_emit_consumes_run_signals_and_flags_report(tmp_path, monkeypatch):
113     """End-to-end: a signal recorded by the segmenter layer is popped by
114     emit_indexer_report, lands in the written report, and does not leak."""
115     monkeypatch.setenv("GEMINI_API_KEY", "x")
116     video = tmp_path / "match4k.mp4"
117     video.write_bytes(b"x")
118     run_signals.incr("md5sig", "oversize_clips_skipped", 7)
119     db = _FakeDB()
120     paths = emit_indexer_report(
121         db=db, video_id="md5sig", video_path=str(video),
122         config={"ai_provider": "gemini", "indexing": {"default_segmenter": "gemini"}},
123         val_results=[_vr("format_check", True)], val_context={"ffprobe_meta":
124             _FFPROBE_OK},
125         segmenter_requested="gemini", segmenter_actual="gemini",
126         was_reingest=False, segmentation_ran=True,

```

```

126     )
127     assert paths is not None
128     with open(paths["json"], encoding="utf-8") as f:
129         assert "oversize_clip_skipped" in f.read()
130     # Consumed: a second pop yields nothing (no cross-run leakage).
131     assert run_signals.pop("md5sig") == {}
132
133
134     def test_motion_segmenter_flags_synthetic_metadata(monkeypatch):
135         monkeypatch.delenv("GEMINI_API_KEY", raising=False)
136         env = _record(segmenter_requested="motion", segmenter_actual="motion",
137                       config={"indexing": {"default_segmenter": "motion"}})
138         codes = _codes(env)
139         assert "synthetic_motion_metadata" in codes
140         # motion is not AI-based -> the provider-noop rule must NOT fire even with no key
141         assert "silent_provider_noop" not in codes
142
143
144     def test_segmenter_divergence_fires_only_without_explicit_override(monkeypatch):
145         monkeypatch.setenv("GEMINI_API_KEY", "x")
146         cfg = {"ai_provider": "gemini", "indexing": {"default_segmenter": "wasb_hybrid"}}
147         # No explicit request, but a different segmenter ran -> a fallback diverged: flag
148         it.
149         env = _record(segmenter_requested=None, segmenter_actual="gemini", config=cfg)
150         assert "segmenter_divergence" in _codes(env)
151         # Explicit --segmenter override that differs from config default is NOT a
152         divergence.
153         env2 = _record(segmenter_requested="gemini", segmenter_actual="gemini", config=cfg)
154         assert "segmenter_divergence" not in _codes(env2)
155
156
157     def test_reingest_info_flag(monkeypatch):
158         monkeypatch.setenv("GEMINI_API_KEY", "x")
159         env = _record(was_reingest=True)
160         assert "reingest_replaced_prior" in _codes(env)
161
162
163     def test_validation_failure_halts_and_flags(monkeypatch):
164         monkeypatch.setenv("GEMINI_API_KEY", "x")
165         env = _record(
166             val_results=[_vr("format_check", True), _vr("relevance_check", False, "court
167             color not found")],
168             play_segments=[],
169             segmenter_actual=None,
170             segmentation_ran=False,
171         )
172         assert env.get_stage("validation").status == "error"
173         assert env.get_stage("segmentation").status == "not_run"
174         assert "validation_failed" in _codes(env)
175         assert env.verdict == "fail"
176
177
178     def test_fps_fallback_flag_when_no_probe_but_segmented(monkeypatch):
179         monkeypatch.setenv("GEMINI_API_KEY", "x")
180         env = _record(val_context={}, segmentation_ran=True)
181         assert env.video_meta.fps_source == "fallback"
182         assert "fps_fallback_used" in _codes(env)
183
184
185     def test_emit_writes_files_next_to_video(tmp_path, monkeypatch):
186         monkeypatch.setenv("GEMINI_API_KEY", "x")
187         video = tmp_path / "match.mp4"
188         video.write_bytes(b"not really a video")
189         db = _FakeDB(play=[{"duration": 4.0, "metadata": {"detector": "wasb-hybrid"}}],
190 candidates=[{"id": 1}])

```

```

187     paths = emit_indexer_report(
188         db=db, video_id="md5abc", video_path=str(video),
189         config={"ai_provider": "gemini", "indexing": {"default_segmenter":
"wasb_hybrid"}}},
190         val_results=[_vr("format_check", True)], val_context={"ffprobe_meta":
_FFPROBE_OK},
191         segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid",
192         was_reingest=False, segmentation_ran=True,
193     )
194     assert paths is not None
195     assert os.path.exists(paths["json"])
196     assert (tmp_path / "match.report.md").exists()
197     assert (tmp_path / "match.summary.md").exists()
198
199
200 def test_emit_accepts_typed_appconfig(tmp_path, monkeypatch):
201     """Reporting must work whether main passes a plain dict or the typed AppConfig
202     (backend.config) ? the reporting layer normalizes AppConfig -> dict."""
203     from backend.config import AppConfig
204
205     monkeypatch.setenv("GEMINI_API_KEY", "x")
206     video = tmp_path / "m.mp4"
207     video.write_bytes(b"x")
208     db = _FakeDB(play=[{"duration": 4.0, "metadata": {"detector": "wasb-hybrid"}}],
candidates=[{"id": 1}])
209     cfg = AppConfig() # typed config object, not a dict
210     paths = emit_indexer_report(
211         db=db, video_id="md5cfg", video_path=str(video), config=cfg,
212         val_results=[_vr("format_check", True)], val_context={"ffprobe_meta":
_FFPROBE_OK},
213         segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid",
214         was_reingest=False, segmentation_ran=True,
215     )
216     assert paths is not None
217     assert os.path.exists(paths["json"])
218
219
220 def test_emit_disabled_via_config(tmp_path, monkeypatch):
221     monkeypatch.setenv("GEMINI_API_KEY", "x")
222     db = _FakeDB()
223     paths = emit_indexer_report(
224         db=db, video_id="md5abc", video_path=str(tmp_path / "m.mp4"),
225         config={"report": {"enabled": False}},
226         val_results=[_vr("format_check", True)], val_context={},
227         segmenter_actual="motion", segmentation_ran=True,
228     )
229     assert paths is None
230

```

41. assistant (2026-06-18T01:29:02.131Z)

The test uses well-formed ``r_frame_rate``. The ``None`` value case for ``r_frame_rate`` is a theoretical edge case but ffprobe practically always emits a string like ``"0/0"`` for unknown rates (not null). The ``.get("r_frame_rate", "0/1")`` default only applies if the key is absent. I'll consider this lower-confidence since ffprobe reliably emits the key as a string. Let me note it but keep looking for more concrete bugs.

Let me look at the test for the gemini provider and verify how ``response.text`` could be ``None``, and the JSON parse path. Let me also check ``test_provider_gemini.py``.

42. user (2026-06-18T01:29:03.374Z)

```

1     import logging
2
3     from unittest.mock import MagicMock, patch

```

```

4     from backend.providers.gemini import GeminiProvider
5
6     def test_gemini_provider_success():
7         # Mock GenAI client
8         with patch("backend.providers.gemini.genai") as mock_genai:
9             client_mock = MagicMock()
10             mock_genai.Client.return_value = client_mock
11
12             # Mock file upload
13             file_mock = MagicMock()
14             file_mock.uri = "gs://dummy"
15             file_mock.name = "dummy_file"
16             file_mock.state.name = "ACTIVE"
17
18             client_mock.files.upload.return_value = file_mock
19             client_mock.files.get.return_value = file_mock
20
21             # Mock generate_content response
22             response_mock = MagicMock()
23             response_mock.text = '["start_time": 1.0, "end_time": 2.0]']
24             client_mock.models.generate_content.return_value = response_mock
25
26             provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
27             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt",
chunk_duration=10.0)
28
29             assert len(segments) == 1
30             assert segments[0]["start_time"] == 1.0
31             assert metrics["upload_time"] >= 0
32             assert metrics["process_time"] >= 0
33             assert metrics["gen_time"] >= 0
34
35             # Ensure delete is called
36             client_mock.files.delete.assert_called_with(name="dummy_file")
37
38     def test_gemini_provider_invalid_json():
39         with patch("backend.providers.gemini.genai") as mock_genai:
40             client_mock = MagicMock()
41             mock_genai.Client.return_value = client_mock
42             file_mock = MagicMock()
43             file_mock.state.name = "ACTIVE"
44             client_mock.files.upload.return_value = file_mock
45             client_mock.files.get.return_value = file_mock
46
47             response_mock = MagicMock()
48             response_mock.text = '{"not_a_list": true}'
49             client_mock.models.generate_content.return_value = response_mock
50
51             provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
52             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
53             assert len(segments) == 0
54
55     def test_gemini_provider_rejects_oversize_before_upload(monkeypatch, caplog):
56         with patch("backend.providers.gemini.genai") as mock_genai:
57             client_mock = MagicMock()
58             mock_genai.Client.return_value = client_mock
59             # 600MB > MAX_UPLOAD_MB (500) -> refuse without ever uploading
60             monkeypatch.setattr("backend.providers.gemini.os.path.getsize",
61                                 lambda p: 600 * 1024 * 1024)
62             provider = GeminiProvider(api_key="", model_name="gemini-flash-latest")
63             with caplog.at_level(logging.ERROR, logger="backend.providers.gemini"):
64                 segments, metrics = provider.analyze_video("big.mp4", "p")
65             assert segments == []

```

```

66         client_mock.files.upload.assert_not_called()
67         # The refusal must be LOUD and machine-visible: callers aggregate the metrics
68         # flag into the per-video report (oversize_clip_skipped rule).
69         assert metrics["oversize_skipped"] is True
70         assert any("SKIPPED WITHOUT ANALYSIS" in r.message for r in caplog.records)
71
72
73     def test_gemini_provider_upload_fail():
74         with patch("backend.providers.gemini.genai") as mock_genai:
75             client_mock = MagicMock()
76             mock_genai.Client.return_value = client_mock
77             client_mock.files.upload.side_effect = Exception("Upload err")
78
79             provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
80             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
81             assert len(segments) == 0
82

```

43. assistant (2026-06-18T01:29:09.961Z)

Now let me look at how the `harvest.\_highlights` computes coverage and the potential division. `dur` is `video\_meta.get("duration\_s")` and the check is `if dur:` so zero-guarded. Good.

Let me dig into the `automated\_provider.py` `submit` more carefully. The `\_labeler` returns `List[Interval]`. Then:

```

```python
intervals = sorted((float(s), float(e)) for s, e in intervals)
```

```

This assumes each item is a 2-tuple. If the labeler returns a 3-tuple or single value, it raises. That's an internal contract though, fine.

Let me look at the gemini segmenter validation path that consumes the parsed segments - specifically how shot_count etc. are accessed, around line 309+.

44. user (2026-06-18T01:29:11.224Z)

```

305         # Populate SQLite DB with parsed play segments
306         segments_data = []
307         logger.info(f"Processing {len(ai_segments)} play segments through
validation...")
308
309         def parse_time(val) -> float:
310             """Converts a timestamp value to total float seconds.
311             Handles float/int pass-through, plain numeric strings, and
312             colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
313             when unexpected formats are encountered."""
314             if isinstance(val, (int, float)):
315                 return float(val)
316             if isinstance(val, str):
317                 val = val.strip()
318                 if ":" in val:
319                     # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal
seconds.
320
321                     # Convert it, but warn so the issue is visible in logs.
322                     parts = val.split(":")
323                     parts.reverse()
324                     total = 0.0
325                     for i, p in enumerate(parts):
326                         try:
327                             total += float(p) * (60 ** i)
328                         except ValueError:
329                             pass
330                     logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")

```

```

330         return total
331     try:
332         return float(val)
333     except ValueError:
334         logger.warning(f"Could not parse timestamp string '{val}' as a
number. Defaulting to 0.0.")
335         return 0.0
336         logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
337         return 0.0
338
339     # 7. Parse timestamps and build candidate segment list
340     idx_cfg = self.config.get("indexing", {})
341     min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
342     max_segment_duration = 90.0 # Singles play segments almost never exceed this
343
344     candidates = []
345     for idx, segment in enumerate(ai_segments):
346         start = parse_time(segment.get("start_time", 0.0))
347         end = parse_time(segment.get("end_time", 0.0))
348         if start >= end:
349             logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
350             continue
351             candidates.append({
352                 "idx": idx,
353                 "start": start,
354                 "end": end,
355                 "raw": segment
356             })
357
358     # 8. Post-hoc validation pass
359     logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
360     validated = []
361     rejected = []
362
363     for c in candidates:
364         label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
365
366         # A: Reject segments that start beyond the video
367         if video_duration > 0 and c["start"] >= video_duration:
368             rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
369             continue
370
371         # B: Clamp end_time to video duration
372         if video_duration > 0 and c["end"] > video_duration:
373             logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
374             c["end"] = video_duration
375
376         dur = c["end"] - c["start"]
377
378         # C: Reject segments shorter than min_segment_duration
379         if dur < min_segment_duration:
380             rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
381             continue
382
383         # D: Flag suspiciously long segments
384         if dur > max_segment_duration:
385             logger.warning(
386                 f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "

```

```

387         f"This may be a detection error. Keeping but flagging."
388     )
389
390     validated.append(c)
391
392     # E: Resolve overlapping segments (keep the longer one)
393     if len(validated) > 1:
394         # Sort by start time

```

45. assistant (2026-06-18T01:29:17.261Z)

This is in the segmenter (not strictly my area, but it consumes the provider output). The parse handling is robust. This is outside my core area anyway.

Let me focus and re-examine the `export.py` `\_tc` for negative time values, since `start\_time`/`end\_time` could be negative in edge cases. With `t = -1.0`: `int(t // 3600)` = `int(-0.000277...)`. ... actually $-1.0 // 3600 = -1.0$, so `h = int(-1.0) = -1`. This would produce malformed timecode but it's not a crash. Segments shouldn't have negative times anyway.

Let me reconsider the slicer's `compute\_clip\_windows` against its test to validate the `"all"` handling and rally id matching.

46. user (2026-06-18T01:29:18.299Z)

```

1     """Tests for the rally slicer's pure logic (no ffmpeg)."""
2     import pytest
3
4     from backend.tools.rally_slicer import (
5         load_rally_labels, compute_clip_windows, ClipWindow, BEGIN_PADDING, END_PADDING,
6     )
7     from backend.tools.export import basic_stats, segments_to_json, segments_to_csv,
segments_to_edl
8
9
10    def _write(tmp_path, rows,
header="rally_number,start_time,end_time,start_mmss,end_mmss"):
11        p = tmp_path / "labels.csv"
12        p.write_text(header + "\n" + "\n".join(rows) + "\n", encoding="utf-8")
13        return str(p)
14
15
16    # ----- load_rally_labels
17
18    def test_load_decimal_and_mmss(tmp_path):
19        csv = _write(tmp_path, [
20            "1,3.0,21.0,,", # decimal
21            "2,,,0:29.000,0:32.000", # mm:ss fallback
22        ])
23        labels = load_rally_labels(csv)
24        assert labels["1"] == pytest.approx((3.0, 21.0))
25        assert labels["2"] == pytest.approx((29.0, 32.0))
26
27
28    def test_load_skips_missing_and_degenerate(tmp_path):
29        csv = _write(tmp_path, [
30            "7.5,106.031,,, ", # MISSED row: no end -> skipped
31            "17,237.0,237.0,,", # zero-duration -> skipped
32            "3,44.5,51.0,,", # good
33        ])
34        labels = load_rally_labels(csv)
35        assert "7.5" not in labels and "17" not in labels
36        assert labels["3"] == pytest.approx((44.5, 51.0))
37
38
39    def test_fractional_ids_preserved(tmp_path):

```



```

40         csv = _write(tmp_path, ["16.5,214.0,223.0,,"])
41         assert "16.5" in load_rally_labels(csv)
42
43
44     # ----- compute_clip_windows
45
46     def test_padding_applied():
47         w = compute_clip_windows({"1": (10.0, 20.0)}, ["1"])
48         assert w == [ClipWindow("1", 10.0 - BEGIN_PADDING, 20.0 + END_PADDING)]
49
50
51     def test_clamp_to_video_bounds():
52         # start near 0 clamps to 0; end past duration clamps to duration
53         w = compute_clip_windows({"1": (0.2, 99.0)}, ["1"], video_duration=99.5)
54         assert w[0].start == 0.0
55         assert w[0].end == 99.5
56
57
58     def test_all_selects_every_rally_in_time_order():
59         labels = {"2": (29.0, 32.0), "1": (3.0, 21.0)}
60         w = compute_clip_windows(labels, ["all"])
61         assert [x.rally for x in w] == ["1", "2"] # ordered by start time, not dict order
62
63
64     def test_subset_and_unknown_skipped():
65         labels = {"1": (3.0, 21.0), "2": (29.0, 32.0), "3": (44.5, 51.0)}
66         w = compute_clip_windows(labels, ["3", "1", "99"]) # 99 unknown -> dropped
67         assert [x.rally for x in w] == ["3", "1"] # preserves requested order
68
69
70     def test_window_entirely_past_end_skipped():
71         w = compute_clip_windows({"1": (200.0, 210.0)}, ["1"], video_duration=100.0)
72         assert w == []
73
74
75     # ----- Value Layer exports
76     (minimal slice)
77
78     def _segs():
79         return [
80             {"id": 1, "start_time": 10.0, "end_time": 15.0, "duration": 5.0, "metadata":
81             {"shot_count": 4, "detector": "wasb-hybrid-gemini"}},
82             {"id": 2, "start_time": 30.0, "end_time": 33.5, "duration": 3.5, "metadata":
83             {"shot_count": 3, "detector": "wasb-hybrid-gemini"}},
84         ]
85
86
87     def test_basic_stats_empty_and_nonempty():
88         assert basic_stats([])["count"] == 0
89         s = basic_stats(_segs())
90         assert s["count"] == 2
91         assert s["total_duration_s"] == 8.5
92         assert s["avg_duration_s"] == 4.25
93
94
95     def test_segments_to_json_roundtrips_and_includes_stats():
96         j = segments_to_json(_segs(), video_id="v1", video_path="clip.mp4")
97         assert j["video_id"] == "v1"
98         assert len(j["segments"]) == 2
99         assert j["stats"]["count"] == 2
100
101
102     def test_segments_to_csv_has_header_and_data():
103         csv = segments_to_csv(_segs())
104         assert csv.startswith("start_time,end_time")

```

```

102         assert "10.0,15.0,5.0,4" in csv
103
104
105     def test_segments_to_edl_produces_plausible_title_and_clip_lines():
106         edl = segments_to_edl(_segs(), title="MyReel")
107         assert "TITLE: MyReel" in edl
108         assert "AX V C" in edl
109         assert "FROM CLIP NAME" in edl
110

```

47. assistant (2026-06-18T01:29:21.925Z)

The slicer logic is well covered. Now let me reconsider the `\_read\_time` in `rally\_slicer.py`. `\_parse\_timestamp` from `backend.eval.annotations`. Let me check what `\_parse\_timestamp` raises for various inputs to ensure `\_read\_time`'s `except ValueError` is correct.

48. user (2026-06-18T01:29:22.750Z)

```

1      """Load hand-annotated ground-truth rally timestamps from CSV.
2
3      Annotation file format (one file per video):
4
5          start,end
6          00:01:12,00:01:39
7          102.5,131.0
8
9      - A header row `start,end` is optional (auto-detected and skipped).
10     - Each timestamp is either decimal seconds (`102.5`) or colon-separated
11       `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
12     - Blank lines and `#` comment lines are ignored.
13     """
14
15     import csv
16     import logging
17     from typing import List, Tuple
18
19     logger = logging.getLogger(__name__)
20
21     Interval = Tuple[float, float]
22
23
24     def parse_timestamp(value: str) -> float:
25         """Parse a timestamp string to float seconds.
26
27         Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
28         Raises ValueError on anything unparseable so bad annotations fail loudly.
29         """
30         value = value.strip()
31         if not value:
32             raise ValueError("empty timestamp")
33         if ":" in value:
34             parts = value.split(":")
35             if len(parts) > 3:
36                 raise ValueError(f"too many ':' segments in timestamp '{value}'")
37             total = 0.0
38             for part in parts: # most-significant first (H, M, S)
39                 total = total * 60 + float(part)
40             return total
41         return float(value)
42
43
44     def load_annotations(csv_path: str) -> List[Interval]:
45         """Load and validate ground-truth rally intervals from a CSV file.
46
47         Delegates to the canonical loader (gt_loader.py, ADR-008 prereq), which accepts BOTH

```

```

48         golden conventions: positional/headered ``start,end`` (the collector export) and the
49         rally-annotator ``rally_number,start_time,end_time,...`` schema. Strict: raises
50         ValueError on malformed rows (bad timestamp, start >= end) so labeling mistakes
51         surface immediately rather than silently skewing scores. Returns intervals sorted by
52         start time.
53         """
54         from backend.eval.gt_loader import load_gt_intervals
55         intervals = load_gt_intervals(csv_path, strict=True)
56         logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
57         return intervals
58
59
60     def write_scaffold(csv_path: str) -> None:
61         """Write an empty annotation file with just the header, for hand-filling."""
62         with open(csv_path, "w", newline="") as f:
63             writer = csv.writer(f)
64             writer.writerow(["start", "end"])
65

```

49. assistant (2026-06-18T01:29:29.582Z)

Note: `parse_timestamp` can raise `ValueError` from `float(part)` for `"00:1a:12"`, which `_read_time` catches. But there's a subtle issue: for a colon value like `"1:2:3:4"` it raises `ValueError` (too many segments) ? caught. Good. The `_read_time` only catches `ValueError`. `parse_timestamp` only raises `ValueError`. Consistent.

Now let me reconsider the `harvest.py` `r_frame_rate` parsing issue more concretely, and also revisit one thing in `_segmentation_stage`: `detector_tag = (play_segments[0].get("metadata") or {}).get("detector")`. If `play_segments[0]["metadata"]` is a JSON string (DB round-trip), `.get` would fail because str has no `.get`. Wait ? `(play_segments[0].get("metadata") or {})` ? if `metadata` is a non-empty string, `"..."` or `{}` returns the string, then `.get("detector")` ? `AttributeError`. Let me verify whether `play_segments` here come from the DB (where `metadata` is a JSON string) or from the segmenter (dict).

Let me check `query_play_segments` to see the metadata type.

50. user (2026-06-18T01:29:30.320Z)

```

backend\main.py:562: def query_play_segments(req: QueryRequest):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-563-     filters = {
backend\main.py-564-         "server": req.server,
backend\main.py-565-         "receiver": req.receiver,
backend\main.py-566-         "duration_min": req.duration_min,
backend\main.py-567-         "event_type": req.event_type
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-568-     }
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-569-     segments =
db_instance.query_play_segments(req.video_id, filters)
backend\main.py-570-     return {"play_segments": segments}
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-571-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\main.py-572- @app.post("/api/compile")
backend\main.py-573- def compile_highlights(req: CompileRequest):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-574-     video =
db_instance.get_video(req.video_id)
backend\main.py-575-     if not video:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-576-         raise
HTTPException(status_code=404, detail="Indexed video record not found.")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-577-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-578-     # Retrieve matching
play segments from db
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-579-     all_segments =
db_instance.query_play_segments(req.video_id, {})
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-580-     matched_segments =
[s for s in all_segments if s["id"] in req.segment_ids]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-581-

```

```

backend\main.py-582-     if not matched_segments:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-583-         raise
HTTPException(status_code=400, detail="No matching play segments found to stitch.")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-584-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-585-         # Reject path
traversal / absolute output names (os.path.join would otherwise let an
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-586-         # absolute or
'..' -laden output_filename write anywhere on disk).
backend\main.py-587-     try:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-588-         out_name =
safe_output_filename(req.output_filename)
backend\main.py-589-     except ValueError as e:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-590-         raise
HTTPException(status_code=400, detail=str(e)) from e
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-591-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-592-         # The configured
shot-count floor (stitching.min_shot_count) applies on the API
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-593-         # path too ? parity
with scripts/run_process_and_stitch.py. Segments WITHOUT shot_count
backend\main.py-594-         # metadata are exempt: the floor filters known counts only, so
explicitly
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-595-         # requested
manual/legacy segments can't silently vanish under the default floor.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-596-         stitching =
getattr(global_config, "stitching", None) or StitchingConfig()
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-597-         kept = []
backend\main.py-598-         for s in matched_segments:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-599-             meta =
s.get("metadata") or {}
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-600-             sc =
meta.get("shot_count") if isinstance(meta, dict) else None
backend\main.py-601-             try:
backend\main.py-602-                 if sc is not None and int(sc) < stitching.min_shot_count:
--
backend\infrastructure\database.py:389:     def query_play_segments(self, video_id: str, filters:
Dict[str, Any]) -> List[Dict[str, Any]]:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-390-
"""
backend\infrastructure\database.py-391-         Dynamically queries play segments based on
filters:
backend\infrastructure\database.py-392-         - duration_min: float
backend\infrastructure\database.py-393-         - server: string
backend\infrastructure\database.py-394-         - receiver: string
backend\infrastructure\database.py-395-         - event_type: string (e.g. smash, foul)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-396-
"""
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-397-
query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-398-
params = [video_id]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-399-
backend\infrastructure\database.py-400-         if filters.get("duration_min"):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-401-
query += " AND r.duration >= ?"
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-402-
params.append(filters["duration_min"])
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-403-
backend\infrastructure\database.py-404-         if filters.get("server"):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-405-
query += " AND r.server = ?"
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-406-
params.append(filters["server"])
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-407-
backend\infrastructure\database.py-408-         if filters.get("receiver"):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-409-

```

```

query += " AND r.receiver = ?"
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-410-
params.append(filters["receiver"])
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-411-
backend\infrastructure\database.py-412-         if filters.get("event_type"):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-413-
# Check if there is an event of a specific type inside this segment
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-414-
query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND e.type = ?)"
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-415-
params.append(filters["event_type"])
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-416-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-417-
segments_list = []
backend\infrastructure\database.py-418-         with self._connect() as conn:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-419-
cursor = conn.cursor()
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-420-
rows = cursor.execute(query, params).fetchall()
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-421-
backend\infrastructure\database.py-422-         for row in rows:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-423-
r_dict = dict(row)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-424-
r_id = r_dict["id"]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-425-
r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-426-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-427-
# Fetch associated events
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-428-
event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?", (r_id,)).fetchall()
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-429-
r_dict["events"] = []
--
tests\test_reporting_harvest.py:22:     def query_play_segments(self, video_id, filters):
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-23-
return list(self._play)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-24-
tests\test_reporting_harvest.py-25-     def query_candidate_segments(self, video_id,
detector=None, only_unlabeled=False):
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-26-
return list(self._candidates)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-27-
tests\test_reporting_harvest.py-28-     def get_video(self, video_id):
tests\test_reporting_harvest.py-29-         return {"id": video_id} if self._existing else None
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-30-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-31-
tests\test_reporting_harvest.py-32-def _vr(name, passed, message="ok"):
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-33-
return ValidationResult(validator_name=name, passed=passed, message=message)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-34-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-35-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-36-_FFPROBE_OK = {
tests\test_reporting_harvest.py-37-     "streams": [{"width": 1920, "height": 1080,
"r_frame_rate": "60000/1001", "codec_name": "hevc"}],
tests\test_reporting_harvest.py-38-     "format": {"format_name": "mov,mp4,m4a", "duration":
"676.0", "size": "5500000000"},
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-39-}
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-40-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-41-
tests\test_reporting_harvest.py-42-def _record(**kw):
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-43-     base
= dict(

```

```

C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-44-
video_id="md5abc",
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-45-
video_path=None,
tests\test_reporting_harvest.py-46-         config={"ai_provider": "gemini", "ai_model": "gemini-
flash", "indexing": {"default_segmenter": "wasb_hybrid"}},
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-47-
val_results=[_vr("format_check", True), _vr("relevance_check", True)],
tests\test_reporting_harvest.py-48-         val_context={"ffprobe_meta": _FFPROBE_OK},
tests\test_reporting_harvest.py-49-         play_segments=[{"duration": 5.0, "metadata":
{"detector": "wasb-hybrid-gemini"}}],
tests\test_reporting_harvest.py-50-         candidates=[{"id": 1}, {"id": 2}],
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-51-
segmenter_requested="wasb_hybrid",
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-52-
segmenter_actual="wasb_hybrid",
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-53-
was_reingest=False,
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-54-
segmentation_ran=True,
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-55-         )
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-56-
base.update(kw)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-57-
return harvest.record_run(**base).envelope
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-58-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-59-
tests\test_reporting_harvest.py-60-def _codes(env):
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-61-
return {f.code for f in env.flags}
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-62-

```

51. assistant (2026-06-18T01:29:39.203Z)

Confirmed: `query\_play\_segments` (database.py line 425) deserializes `metadata` to a dict via `json.loads`. So in `harvest.\_segmentation\_stage`, `play\_segments` from the DB always have `metadata` as a dict. The `(play\_segments[0].get("metadata") or {}).get("detector")` is safe in production. Not a bug.

Now let me reconsider the `r\_frame\_rate` None case once more. ffprobe JSON output: for streams that have a frame rate, `r\_frame\_rate` is a string like `"30/1"`. It's a string, not null, for video streams. The default `"0/1"` only triggers when the key is missing entirely. So the `None.split` crash isn't realistically reachable. I'll drop it as too speculative.

Let me look at the `\_segment\_metadata` in export.py and `format\_rally\_summary`'s `total\_dur` computation more carefully for a real bug. In `format\_rally\_summary`:

```

```python
total_dur = sum(
 float(s.get("duration", float(s.get("end_time", 0)) - float(s.get("start_time", 0))))
 for s in segs
)
...

```

If `duration` key exists but is `None` (segment with explicit null duration), `s.get("duration", default)` returns `None` (because the key exists), then `float(None)` ? `TypeError`. Is that reachable? The default is only used if key absent. Let me check whether segments can have `duration: None`. In the DB, the column is likely NOT NULL. Let me not over-speculate; segments from segmenters always set duration. Lower confidence.

Let me now look at a potentially more interesting area ? let me check `test\_annotation\_providers.py` and `test\_automated\_provider.py` to verify the contract claims in `base.py` about `fetch` returning sorted intervals.

## 52. user (2026-06-18T01:29:40.160Z)

```

1 import pytest

```

```

2
3 from backend.annotations import annotation_registry, AnnotationProvider
4 from backend.annotations.base import DONE, FAILED
5 from backend.annotations.file_provider import FileProvider
6
7
8 def _write_csv(path) -> str:
9 # Mixed decimal + HH:MM:SS forms; header auto-skipped by load_annotations.
10 path.write_text("start,end\n0.0,3.0\n00:00:05,00:00:09\n")
11 return str(path)
12
13
14 # ----- registry -----
15
16 def test_registry_has_file_provider():
17 assert "file" in annotation_registry.list_available()
18 p = annotation_registry.get("file")
19 assert isinstance(p, AnnotationProvider)
20 assert p.name == "file"
21
22
23 def test_registry_unknown_provider_raises():
24 with pytest.raises(KeyError):
25 annotation_registry.get("does-not-exist")
26
27
28 def test_registry_get_passes_kwargs(tmp_path):
29 p = annotation_registry.get("file", csv_dir=str(tmp_path))
30 assert isinstance(p, FileProvider)
31 assert p.csv_dir == str(tmp_path)
32
33
34 # ----- FileProvider: submit / poll / fetch -----
35
36 def test_file_provider_direct_path(tmp_path):
37 csv = _write_csv(tmp_path / "rallies.csv")
38 p = FileProvider()
39 job = p.submit(csv)
40 assert job == csv
41 assert p.poll(job) == DONE
42 assert p.fetch(job) == [(0.0, 3.0), (5.0, 9.0)]
43
44
45 def test_file_provider_dir_resolution(tmp_path):
46 _write_csv(tmp_path / "vid123.csv")
47 p = FileProvider(csv_dir=str(tmp_path))
48 job = p.submit("vid123") # bare video_id, resolved to <dir>/vid123.csv
49 assert job.endswith("vid123.csv")
50 assert p.poll(job) == DONE
51 assert len(p.fetch(job)) == 2
52
53
54 def test_file_provider_missing_file_is_failed(tmp_path):
55 p = FileProvider(csv_dir=str(tmp_path))
56 job = p.submit("nonexistent")
57 assert p.poll(job) == FAILED
58 with pytest.raises(FileNotFoundError):
59 p.fetch(job)
60
61
62 def test_file_provider_annotate_convenience(tmp_path):
63 csv = _write_csv(tmp_path / "rallies.csv")
64 p = FileProvider()
65 assert p.annotate(csv) == [(0.0, 3.0), (5.0, 9.0)]
66

```

```

67
68 def test_annotate_raises_when_not_done(tmp_path):
69 # annotate() on a synchronous provider with a missing file must fail loudly,
70 # not silently return [] (which would skew downstream regression scores).
71 p = FileProvider(csv_dir=str(tmp_path))
72 with pytest.raises((RuntimeError, FileNotFoundError)):
73 p.annotate("nonexistent")
74
75
76 # ----- the contract that makes the whole flow work: provider output == GT shape -----
77
78 def test_provider_output_matches_metrics_interval_shape(tmp_path):
79 """A provider's fetch() output must be directly scorable by the eval harness ?
80 same List[(float, float)] shape metrics.score() expects. This is the keystone
81 that lets one scorer grade product, vendor, and annotator-vs-annotator."""
82 from backend.eval import metrics
83 csv = _write_csv(tmp_path / "gt.csv")
84 gt = FileProvider().annotate(csv)
85 # Feed provider output straight into the scorer as ground truth ? must not raise.
86 s = metrics.score(preds=[(0.0, 3.0), (5.0, 9.0)], gts=gt, iou_threshold=0.5)
87 assert s.true_positives == 2
88 assert s.precision == 1.0 and s.recall == 1.0
89

```

### 53. user (2026-06-18T01:29:40.782Z)

```

1 """Tests for the GS-4 AutomatedProvider interface stub: registry wiring, the
2 injected-labeler contract, the not-configured guard, the oracle-lineage warning,
3 and the end-to-end path through the GS-2 regression runner."""
4 import logging
5 import pytest
6
7 from backend.annotations import annotation_registry
8 from backend.annotations.base import DONE, FAILED
9 from backend.annotations.automated_provider import (
10 AutomatedProvider, warn_if_same_lineage,
11)
12
13
14 def _fake_labeler(video_ref, guideline, sport):
15 # Deterministic, model-free stand-in for a real oracle.
16 return [(20.0, 30.0), (0.0, 10.0)] # intentionally unsorted -> provider sorts
17
18
19 # ----- registry -----
20
21 def test_registry_has_auto():
22 assert "auto" in annotation_registry.list_available()
23 p = annotation_registry.get("auto")
24 assert isinstance(p, AutomatedProvider)
25 assert p.name == "auto"
26
27
28 def test_registry_get_injects_labeler_and_lineage():
29 p = annotation_registry.get("auto", labeler=_fake_labeler, lineage="fake")
30 assert p.lineage == "fake"
31 out = p.annotate("vid.mp4")
32 assert out == [(0.0, 10.0), (20.0, 30.0)] # sorted by start
33
34
35 # ----- submit / poll / fetch -----
36
37 def test_submit_poll_fetch_cycle():
38 p = AutomatedProvider(labeler=_fake_labeler, lineage="fake")
39 job = p.submit("vid.mp4")

```



```

40 assert p.poll(job) == DONE
41 assert p.fetch(job) == [(0.0, 10.0), (20.0, 30.0)]
42
43
44 def test_poll_unknown_job_is_failed():
45 p = AutomatedProvider(labeler=_fake_labeler)
46 assert p.poll("never-submitted") == FAILED
47
48
49 def test_fetch_unknown_job_raises():
50 p = AutomatedProvider(labeler=_fake_labeler)
51 with pytest.raises(KeyError):
52 p.fetch("never-submitted")
53
54
55 # ----- the not-configured guard (fail loud, not silent-empty) -----
56
57 def test_unconfigured_labeler_raises():
58 p = annotation_registry.get("auto") # no labeler injected
59 with pytest.raises(NotImplementedError):
60 p.annotate("vid.mp4")
61
62
63 # ----- oracle rule encoded in code -----
64
65 def test_warn_if_same_lineage_true_and_logs(caplog):
66 with caplog.at_level(logging.WARNING):
67 assert warn_if_same_lineage("gemini", "gemini") is True
68 assert any("blind" in r.message.lower() or "lineage" in r.message.lower()
69 for r in caplog.records)
70
71
72 def test_warn_if_same_lineage_false_when_distinct():
73 assert warn_if_same_lineage("gemini", "actionformer") is False
74 assert warn_if_same_lineage(None, "actionformer") is False
75
76
77 # ----- end-to-end through the GS-2 regression runner -----
78
79 def test_regress_with_auto_tier(tmp_path):
80 from backend.infrastructure.database import Database
81 from backend.eval import regression
82
83 d = Database(str(tmp_path / "t.db"))
84 d.add_video("vid1", "/x/m.mp4", "processed", [])
85 d.add_play_segment("vid1", 0.0, 10.0)
86 d.add_play_segment("vid1", 20.0, 30.0)
87
88 # The 'auto' oracle (fake labeler) supplies ground truth; predictions match ? clean.
89 r = regression.regress_with_provider(
90 d, video_id="vid1", video_ref="vid1", tier="auto",
91 baseline_path=str(tmp_path / "b.json"),
92 provider_kwargs={"labeler": _fake_labeler, "lineage": "fake"},
93)
94 assert r.precision == 1.0 and r.recall == 1.0
95 assert r.regressed is False
96

```

#### 54. assistant (2026-06-18T01:30:01.718Z)

Now I have thorough coverage of the tested behavior. Let me examine `reporting/\_\_init\_\_.py`  
`emit\_indexer\_report` one more time. There's an important behavior: `signals =  
run\_signals.pop(video\_id)` happens, then `harvest.record\_run(...)` is only called if obsreport  
available. Fine.

But wait ? there's a subtle concern: in `\_ai\_stage` (harvest.py),  
`run\_signals.get("oversize\_clips\_skipped", 0)` is read, but the segmenter `gemini.py` records it  
under key `"oversize\_clips\_skipped"` via `run\_signals.incr(video\_id, "oversize\_clips\_skipped",  
...)`. Match confirmed.

Let me now look very carefully at `harvest.py` line 56: `num, den = (int(x) for x in  
fps\_str.split("/"))`. The generator with exactly-2 unpacking. If `fps\_str = "30/1"` ? fine. If  
`fps\_str = "30000/1001"` ? fine. But `int("30000")` works. However `int(x)` for a value like  
`"24000/1001"`... fine. What about a fractional like ffprobe avg\_frame\_rate `"0/0"` ? `num=0,  
den=0`, then `if den:` is False, so fps not set, fps\_source stays unknown. No crash. Good.

Let me now turn to a different angle: the `GeminiProvider` retry logic for upload. Look at lines  
65-77. If all 3 attempts fail, `video\_file` stays None ? returns. Good. But there's a subtle  
issue: if `self.client.files.upload` succeeds but raises later... no.

Let me reconsider the `poll\_until` interaction with `\_processed`. The closure mutates  
`video\_file` via `nonlocal`. If `files.get` raises during polling, `poll\_until` does NOT catch  
it ? `check()` raises, propagating out of `poll\_until`. The gemini code catches it at line 96  
`except Exception as e`. But here's the catch: at that point `video\_file` may have been  
reassigned to a partially-fetched state or still hold the old value. Then  
`self.\_safe\_delete(video\_file)` is called ? safe. OK.

Let me look at whether there's a real issue with the FAILED-state path. When `\_processed`  
returns the file for a FAILED state, line 100 reads `video\_file.error.message`. If `error` is  
None (file failed but no error object)... that's a possible AttributeError. But this is  
defensive territory; the genai SDK populates `error` for FAILED. Lower confidence.

Let me step back and broaden the search across the area for any obvious sign/operator bugs I  
might have missed. Let me re-read `harvest.py` `\_highlights` coverage and the  
`\_validation\_stage` status logic.

```
In `_validation_stage`:
```python  
failures = [r for r in val_results if not getattr(r, "passed", True)]  
```
```

Default `True` means: if a result lacks `passed`, it's treated as passed (not a failure). And  
`passed = sum(1 for r in val\_results if getattr(r, "passed", False))` ? default `False`. So a  
result missing `passed` is counted as NOT passed in the count, but NOT a failure in the failures  
list. Minor inconsistency but both defaults are defensible and results always have `passed`. Not  
a bug.

Let me check the `run\_signals` registry once more ? there's a thread-safety concern but it's  
locked. Fine.

Let me look at one thing I glossed over: the gemini provider's response when `run\_bounded`  
returns but `response.text` is None. Line 126: `raw\_text = response.text`. If the model returns  
no text (e.g., safety block), `response.text` could raise or be None. Line 133 guards `if not  
raw\_text`. If `.text` raises an `AttributeError`/`ValueError` (the genai SDK raises if no  
candidates), it's caught at line 129 `except Exception`. Good.

I've now thoroughly traced the entire area. Let me do one final check on the `export.py`  
`segments\_to\_csv` for a NaN/None edge in shot\_count and the `\_tc` nested function being  
redefined per-iteration (perf, not bug).

Let me verify the one remaining candidate I find most concrete: re-examine  
`format\_rally\_summary`'s duration default evaluation more carefully because the default  
expression is **\*\*eagerly evaluated\*\***.

## 55. user (2026-06-18T01:30:03.220Z)

```
56 segs = _to_list(segments)
57 total_dur = sum(
58 float(s.get("duration", float(s.get("end_time", 0)) - float(s.get("start_time",
0))))
59 for s in segs
```

```

60)
61 header = (
62 f"=== {title}: {len(segs)} rallies, "
63 f"{_fmt_hms(total_dur)} ({total_dur:.1f}s) total play ==="
64)
65 if not segs:
66 return header + "\n (no rallies detected)"
67
68 lines = [header, f"{'#':>3} {'start':<8} {'end':<8} {'dur(s)':>7} {'shots':>6}"]
69 for i, s in enumerate(segs, start=1):
70 start = float(s.get("start_time", 0.0))
71 end = float(s.get("end_time", start))
72 dur = float(s.get("duration", end - start))
73 shot = _segment_metadata(s).get("shot_count")
74 shot_str = "-" if shot is None else str(shot)
75 lines.append(
76 f"{i:>3} {_fmt_hms(start):<8} {_fmt_hms(end):<8} {dur:>7.1f}
77 {shot_str:>6}"
78)
79 return "\n".join(lines)
80

```

## 56. assistant (2026-06-18T01:30:14.160Z)

These `.get("duration", <fallback>)` patterns crash only if the key exists with value `None`, which doesn't happen for real segments (DB and segmenter always set numeric duration). Too speculative to report.

I've now traced every file in my assigned area (``backend/providers/`, `backend/sports/`, `backend/annotations/`, `backend/reporting/`, `backend/tools/``) and their callers and tests. The code is mature and well-hardened:

- ``gemini.py`` ? oversize guard, bounded upload retry, bounded poll, bounded generate, safe-delete cleanup, JSON parse guards. All paths return ``([], metrics)`` cleanly on failure. No reachable crash found.
- ``providers/base.py``, ``providers/__init__.py`` ? clean registry, `KeyError` on unknown.
- ``sports/`` ? pure prompt/config returns.
- ``annotations/`` ? submit/poll/fetch contracts well-tested, fetch raises (not silent-empty) on missing.
- ``reporting/harvest.py`` + ``run_signals.py`` + ``__init__.py`` ? defensive throughout, soft obsreport dep, signal keys consistent (``oversize_clips_skipped``), DB metadata is always a dict by the time harvest sees it.
- ``tools/rally_slicer.py``, ``tools/export.py`` ? pure logic, well covered, `ffprobe/ffmpeg` errors caught.

The candidate edge cases I considered (``r_frame_rate=None``, ``duration=None``, ``video_file.error=None``) are not reachable with realistic inputs and have no triggering call path I could construct. Per the precision-over-recall mandate, I'm dropping them rather than manufacturing findings.

Returning an empty candidates list ? the respected, valid answer for a hardened area with no real reachable defect.

## 57. user (2026-06-18T01:30:14.464Z)

Structured output provided successfully

## 58. assistant (2026-06-18T01:30:26.559Z)

I traced every file in my assigned area along with its callers and tests, and found no real, reachable defect that a maintainer would agree is a bug.

Files examined (all in ``C:/Users/avidu/Projects/badminton-highlight-`

indexer/.claude/worktrees/hunt-base/`):

- `backend/providers/gemini.py`, `backend/providers/base.py`, `backend/providers/\_\_init\_\_.py`
- `backend/sports/badminton.py`, `backend/sports/base.py`
- `backend/annotations/base.py`, `backend/annotations/automated\_provider.py`,  
`backend/annotations/file\_provider.py`
- `backend/reporting/harvest.py`, `backend/reporting/run\_signals.py`,  
`backend/reporting/\_\_init\_\_.py`
- `backend/tools/rally\_slicer.py`, `backend/tools/export.py`

Cross-checked against callers/contracts: `backend/infrastructure/execution\_policy.py` (bounded poll/retry used by Gemini), `backend/infrastructure/database.py::query\_play\_segments` (confirms `metadata` is deserialized to a dict before `harvest` sees it),  
`backend/pipeline/segmenters/gemini.py` (confirms the `oversize\_clips\_skipped` signal key matches between producer and `harvest.\_ai\_stage`/`run\_signals`),  
`backend/eval/annotations.py::parse\_timestamp` (confirms `\_read\_time`'s `except ValueError` matches the only exception it raises), and the corresponding test files.

Edge cases I specifically considered and deliberately dropped as not reachable with realistic inputs (no concrete trigger path I could construct):

- `harvest.py` `r\_frame\_rate` parsing crashing on a literal `None` value ? ffmpeg emits this field as a string for video streams; the `None.split` path requires the key to be present-but-null, which doesn't occur.
- `export.py`/`harvest.py` `.get("duration", fallback)` raising on an explicit `duration: None` ? segments always carry numeric durations from both the segmenter and the DB.
- `gemini.py` reading `video\_file.error.message` when `error` is None on a FAILED file ? the genai SDK populates `error` for FAILED state.

Per the precision-over-recall mandate, returning an empty candidates list. The area is mature and hardened (bounded waits, oversize guards, safe-delete cleanup, fail-loud annotation contracts, soft obsreport dependency), and I would rather report nothing than manufacture a finding.